Control de Fortran. Grupo CLI01, luns, 14 decembro, 2009

Escribe un programa en Fortran que faga o seguinte:

1. (**2 PUNTOS**). Ler por teclado un número enteiro n. Debes verificar que $n > 2$ e $n < 10$ e, se non é así, volver a pedilo por teclado.

2. (**1 PUNTO**). Reservar memoria dinámicamente para dúas matrices a e b (ambas reais de orde n), e ler a por teclado. Podes usar esta matriz como exemplo ($n = 3$):

$$a = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

3. (**4 PUNTOS**). Calcula-la matriz b de modo que o elemento b_{ij} sexa a suma dos elementos da matriz a agás o elemento a_{ij}. Ademais, mostra-la matriz b por pantalla (cada fila nunha liña distinta).

4. (**3 PUNTOS**). Chamar a un subprograma `escribe_matriz(...)` (debes decidi-lo seu tipo e argumentos) que escriba no arquivo `matriz.dat` a matriz b (cada fila nunha liña distinta do arquivo).

```
program cli01
real, dimension(:,:), allocatable::a, b

!Le a dimension da matriz
do
  print*, "Introduce dimensions da matriz"
  read*, n
  if (n > 2 .and. n < 10) exit
end do


!Reserva memoria
allocate(a(n,n), b(n,n))

!Le a matriz a
print*, "Introduce matriz a"
do i=1,n
  read*, (a(i,j), j=1,n)
end do

print*, "Matriz a"
do i=1,n
  print*, (a(i,j), j=1,n)
end do

! Calculo da matriz b
do i=1,n
```

```
  do j=1,n
    b(i,j)=0
    do k=1,n
      do l=1,n
        b(i,j)=b(i,j)+a(k,l)
      end do
    end do
    b(i,j)=b(i,j)-a(i,j)
  end do
end do

!Visualiza a matriz
print*, "Matriz b"
do i=1,n
  print*, (b(i,j), j=1,n)
end do

! Gardar matriz b nun arquivo
call escribe_matriz(b, n)

!Liberar memoria
deallocate(a, b)

stop
end program cli01



subroutine escribe_matriz(x, m)
real, dimension(m,m), intent(in)::x
integer, intent(in)::m

open(1, file="datos.dat", err=1)
do i=1,m
  write(1,*)(x(i,j), j=1,m)
end do
close(1)
return
1 print*, "Erro abrindo o arquivo datos.dat"
stop
end subroutine escribe_matriz
```

Control de Fortran. Grupo CLI02, luns, 14 decembro, 2009

Escribe un programa en Fortran que faga o seguinte:

1. (**2 PUNTOS**). Ler por teclado un número enteiro n. Debes verificar que $n > 2$ e $n < 10$ e, se non é así, volver a pedilo por teclado.

2. (**1 PUNTO**). Reservar memoria dinámicamente para dúas matrices a e b (ambas reais de orde n), e ler a por teclado. Podes usar esta matriz como exemplo ($n = 3$):

$$a = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

3. (**4 PUNTOS**). Calcula-la matriz b de modo que o elemento b_{ij} sexa a suma dos elementos da matriz a agás o elemento a_{ij}. Ademais, mostra-la matriz b por pantalla (cada fila nunha liña distinta).

4. (**3 PUNTOS**). Chamar a un subprograma `escribe_matriz(...)` (debes decidi-lo seu tipo e argumentos) que escriba no arquivo `matriz.dat` a matriz b (cada fila nunha liña distinta do arquivo).

```
program cli02
real, dimension(:,:), allocatable::a, b

!Le a dimension da matriz
do
  print*, "Introduce dimensions da matriz"
  read*, n
  if (n > 2 .and. n < 10) exit
end do


!Reserva memoria
allocate(a(n,n), b(n,n))

!Le a matriz a
print*, "Introduce matriz a"
do i=1,n
  read*, (a(i,j), j=1,n)
end do

print*, "Matriz a"
do i=1,n
  print*, (a(i,j), j=1,n)
end do

! Calculo da matriz b
do i=1,n
```

```
  do j=1,n
    b(i,j)=0
    do k=1,n
      do l=1,n
        b(i,j)=b(i,j)+a(k,l)
  end do
    end do
    b(i,j)=b(i,j)-a(i,j)
  end do
end do

!Visualiza a matriz
print*, "Matriz b"
do i=1,n
  print*, (b(i,j), j=1,n)
end do

! Gardar matriz b nun arquivo
call escribe_matriz(b, n)

!Liberar memoria
deallocate(a, b)

stop
end program cli01

subroutine escribe_matriz(x, m)
real, dimension(m,m), intent(in)::x
integer, intent(in)::m

open(1, file="datos.dat", err=1)
do i=1,m
  write(1,*)(x(i,j), j=1,m)
end do
close(1)
stop
1 print*, "Erro abrindo o arquivo datos.dat"
end subroutine
```

Control de Fortran. Grupo CLI04, mércores, 16 decembro, 2009

Escribe un programa en Fortran que faga o seguinte:

1. (**2 PUNTOS**). Declarar un vector estático **v** de 10 elementos. Ler repetidamente por teclado números enteiros. A lectura debe rematar cando o usuario introduza o número -1 ou ben cando se introduza o décimo número. Almacenar na variábel n o número de valores introducidos.

2. (**2 PUNTOS**). Imprimir por pantalla os números introducidos que son pares (pista: a función `mod(x, y)`, sendo `x` e `y` valores enteiros, retorna o resto da división enteira de `x` entre `y`).

3. (**3 PUNTOS**). Reservar memoria para unha matriz enteira a cadrada de orde n. Calcula-los seus elementos segundo a expresión:

$$a_{ij} = v_i + \sum_{k=1}^{n-j} v_{j+k}, \qquad i, j = 1, \ldots, n \tag{1}$$

   Imprimi-la matriz a por pantalla (cada fila nunha liña distinta).

4. (**3 PUNTOS**). Chamar a un subprograma `escribe_matriz(...)` (debes decidi-lo seu tipo e argumentos) que escriba no arquivo `matriz.dat` a matriz a (cada fila nunha liña distinta do arquivo).

```
program cli04
integer, dimension(10)::v
real, dimension(:,:), allocatable::a

!Le vector
print*, "Introduce elementos do vector "
n=0
do i=1,10
  read*, x
  if (x == -1 .or. n > 9) exit
  n=n+1
  v(i)=x
end do

print*, "Vector v "
print*, (v(i), i=1,n)

do i=1,n
  if (mod(v(i), 2)==0) then
    print*, "valor par", v(i)
  else
    print*, "valor impar", v(i)
  end if
end do
```

```
!Reserva memoria a
allocate(a(n,n))

do i=1,n
  do j=1,n
    a(i,j)=v(i)
    do k=1, n-j
      a(i,j)=a(i,j)+ v(j+k)
    end do
  end do
end do

!Visualiza a matriz
print*, "Matriz a"
do i=1,n
  print*, (a(i,j), j=1,n)
end do

! Gardar matriz b nun arquivo
call escribe_matriz(a, n)

!Liberar memoria
deallocate(a)

stop
end program cli04



subroutine escribe_matriz(x, m)
real, dimension(m,m), intent(in)::x
integer, intent(in)::m

open(1, file="matriz.dat", err=1)
do i=1,m
  write(1,*)(x(i,j), j=1,m)
end do
close(1)
stop
1 print*, "Erro abrindo o arquivo datos.dat"
end subroutine
```

Control de Fortran. Grupo CLI05, martes, 15 decembro, 2009

Escribe un programa en Fortran que faga o seguinte:

1. (**3 PUNTOS**). Calcular e mostrar por pantalla os valores da función $f(x) = e^{-x^2/10} sin(x^2)$ no intervalo $[-5, 5]$ usando un paso $\Delta x = 0{,}01$.

2. (**2 PUNTOS**). Calcular e mostrar por pantalla os valores máximo e mínimo da función $f(x)$ neste intervalo.

3. (**2 PUNTOS**). Calcular e mostrar por pantalla o valor medio da función $f(x)$ neste intervalo, definido por (n é o número de puntos x_i para os que se calcula o valor da función):

$$\langle f \rangle = \frac{1}{n} \sum_{i=1}^{n} f(x_i) \tag{1}$$

4. (**3 PUNTOS**). Chamar a un subprograma `escribe_arquivo(...)` (debes decidi-lo seu tipo e argumentos) que cree o arquivo `arquivo.dat` e escriba nel os valores máximo, mínimo e medio calculados nos apartados anteriores.

```
program cli05
real:: minimo, maximo, medio
a=-5
h=0.01
n=0
minimo=10000
maximo=-200
do
  f=exp(-a*a/10)*sin(a*a)
  if (f<minimo) minimo=f
  if (f>maximo) maximo=f
  medio=medio+f
  n=n+1
  a=a+h
  if (a>5) exit
end do

print*, "Valor maximo ", maximo
print*, "Valor minimo ", minimo
print*, "Valor medio ", medio/n

! Gardar matriz b nun arquivo
call escribe_matriz(maximo, minimo, medio/n)


stop
end program cli05
```

```
subroutine escribe_matriz(x, y,z)
real, intent(in)::x, y,z

open(1, file="datos.dat", err=1)
write(1,*) "Valor maximo ", x
write(1,*) "Valor minimo ", y
write(1,*) "Valor medio ", z
close(1)
stop
1 print*, "Erro abrindo o arquivo datos.dat"
end subroutine
```

Control Fortran

Escribe un programa en Fortran que lea por teclado un número enteiro n, reserve dinámicamente memoria para un vector enteiro $\mathbf{v}$ de dimensión n e lea por teclado as súas compoñentes. Logo, debe chamar a un subprograma `calcula_matriz(...)` (debes decidi-lo seu tipo e argumentos) que calcule unha matriz cadrada de orde n tal que a súa **diagonal secundaria** sexa igual ao vector $\mathbf{v}$. Os restantes elementos da matriz deben te-lo valor -1. Finalmente, dende o programa principal debes chamar a outro subprograma `almacena_matriz(...)` (debes decidi-lo seu tipo e argumentos) que cree o arquivo `matriz.dat` e almacene nel a matriz (cada fila debe escribirse nunha liña distinta do arquivo). Proba con $n = 4$, $\mathbf{v} = [1, 2, 3, 4]$; debes obte-la matriz:

$$\begin{bmatrix} -1 & -1 & -1 & 1 \\ -1 & -1 & 2 & -1 \\ -1 & 3 & -1 & -1 \\ 4 & -1 & -1 & -1 \end{bmatrix}$$

SOLUCIÓN:

```
program diagonal_secundaria

real, dimension(:), allocatable::v
real, dimension(:,:), allocatable::a
print*, "Introduce un numero: "
read*, n
allocate(v(n), a(n,n))
print*, "Introduce vector: "
read*, (v(i), i=1,n)
call calcula_matriz(v, a, n)
call almacena_matriz(a, n)
deallocate(v, a)
stop
end program diagonal_secundaria

! Definicion de calcula_matriz
subroutine calcula_matriz(x, b, m)
real, dimension(m), intent(in)::x
real, dimension(m,m), intent(out)::b
integer, intent(in)::m
do i=1, m
  do j=1,m
    if (i+j == m+1) then
      b(i,j)=x(i)
    else
      b(i,j)=-1
    end if
  end do
end do
return
end subroutine calcula_matriz
```

```
!Definicion de almacena_matriz
subroutine almacena_matriz(c, m)
real, dimension(m,m), intent(in)::c
integer, intent(in)::m
open(1, file="matriz.dat", status="new", err=3)
do i=1, m
  write(1,*) (c(i,j), j=1,m)
end do
close(1)
return
3 print*, "Erro abrindo matriz.dat"
end subroutine almacena_matriz
```

Control Fortran

Escribe un programa en Fortran que lea por teclado un número enteiro n, reserve dinámicamente memoria para unha matriz enteira **m** cadrada de orde n, e lea por teclado as súas compoñentes. Logo debe construir dúas matrices de orde n: 1) unha matriz **a** coas compoñentes pares de **m** (os restantes elementos deben ser 0); b) outra matriz **b** coas compoñentes impares de **m** (os restantes elementos deben ser 0). Finalmente, débese chamar a un subprograma `almacena_matriz(...)` (debes decidi-lo seu tipo e argumentos), que cree o arquivo `matriz_exame.dat` e almacene nel as dúas matrices **a** e **b** (unha fila en cada liña do arquivo). **EXEMPLO**: Proba con $n = 3$ e a matriz:

$$\mathbf{m} = \begin{bmatrix} 3 & 2 & 4 \\ 6 & 1 & 5 \\ 3 & 7 & 2 \end{bmatrix}$$

Debes obte-las seguintes matrices:

$$\mathbf{a} = \begin{bmatrix} 0 & 2 & 4 \\ 6 & 0 & 0 \\ 0 & 0 & 2 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 1 & 5 \\ 3 & 7 & 0 \end{bmatrix}$$

SOLUCIÓN:

```
program dec_2010
integer, dimension(:,:), allocatable::m, a, b

print*, 'Introduce un numero: '
read*, n

allocate(m(n,n), a(n,n), b(n,n))
print*, 'Introduce matriz: '
do i=1,n
  read*, (m(i,j), j=1,n)
end do

do i=1,n
  do j=1,n
    if (mod(m(i,j), 2) == 0) then
      a(i,j)=m(i,j); b(i,j)=0
    else
      a(i,j)=0; b(i,j)=m(i,j)
    end if
  end do
end do
call almacena_matriz(a,b,n)
deallocate(m, a, b)
stop
end program dec_2010

! Define a subroutina almacena_matriz
subroutine almacena_matriz(a, b, n)
```

```
integer, dimension(n,n), intent(in)::a, b
integer, intent(in)::n
open(1, file="matriz_exame.dat", err=3)
do i=1,n
  write(1,*) (a(i,j), j=1,n)
end do
write (1, *) "----------------"
do i=1,n
  write(1,*) (b(i,j), j=1,n)
end do
close(1)
return
3 print*, "Erro abrindo matriz_exame.dat"
end subroutine almacena_matriz
```

Control Fortran

Escribe un programa en Fortran lea polo teclado o número n de elementos do vector dinámico de enteiros **x** e realice as seguintes operacións:

- Ler do teclado o vector **x**.
- Escribe un subprograma chamado `valores_repetidos(...)` (decide o tipo e argumentos) que calcule as secuencias de números iguais dentro do vector, proporcionando o número que se repite e o número de veces que se repite. Por exemplo, para o vector $x = (1, 1, 1, 2, 3, 1, 3, 3, 4, 1, 1)$, con $n = 11$, a saída sería que se repiten os números 1,3 e 1 un número de veces 3,2 e 2 respectivamente.
- O subprograma `valores_repetidos()` debe almacenar no arquivo `secuencias.txt` en liñas distintas os seguintes datos: número de elementos de **x**, vector **x**, e para cada secuencia, indicar o elemento que se repite e número de veces que se repite. Por exemplo, para o vector anterior, o arquivo `secuencias.txt` contén:

```
11
1 1 1 2 3 1 3 3 4 1 1
1 3
3 2
1 2
```

SOLUCIÓN:

```
program control1
integer, dimension(:), allocatable :: x
print '("introduce n: ",$)'
read *, n
allocate(x(n))
print '("introduce x: ",$)'
read *, (x(i),i=1,n)
call valores_repetidos(x,n)
deallocate(x)
stop
1 print *, "o arquivo secuencias.txt xa existe"
stop
end program control1

!---------------------------
subroutine valores_repetidos(x, n)
integer, dimension(n), intent(in) :: x
integer, intent(in) :: n
n_rep = 1
open(1,file="secuencias.txt",status="new",err=1)
write(1,*) n
write(1,*) x
do i=2,n
  if(x(i) == x(i-1)) then
    n_rep = n_rep + 1
  else
    if(n_rep > 1) then
      print *, "o numero", x(i-1), "repetiuse", n_rep, "veces"
      write (1,*) x(i-1), n_rep
      n_rep = 1
    end if
  end if
end do
if(n_rep > 1) then
  print *, "o numero", x(n), "repetiuse", n_rep, "veces"
```

```
  write (1,*) x(n), n_rep
end if
close(1)
return
1 print *, "o arquivo secuencias.txt xa existe"
stop
end subroutine valores_repetidos
```

Control Fortran

Escribe un programa en Fortran que lea do arquivo `matriz.dat` unha matriz cadrada **a** de orde 4 e realice as seguintes operacións:

- Escribe un subprograma chamado `calculos(...)` (decide o tipo e argumentos) que calcule o producto escalar da diagonal principal e a diagonal secundaria.
- Dende o programa principal invoca o subprograma `calculos()` para a matriz **a**.
- Almacena ó final do arquivo `matriz.dat` unha mensaxe indicando se o producto escalar e positivo, negativo ou cero.

EXEMPLO. Proba co seguinte arquivo `matriz.dat`:

$$\mathbf{a} = \begin{bmatrix} 1 & 0 & 2 & -1 \\ 3 & -4 & 1 & 7 \\ 0 & -3 & 8 & 1 \\ 5 & 3 & 7 & -2 \end{bmatrix}$$

e o resultado será $p = -39$.

SOLUCIÓN:

```
program control2
integer, dimension(4,4) :: a
integer :: calculos
open(1,file="matriz.dat",status="old",err=1)
do i=1,4
  read (1,*) (a(i,j),j=1,4)
end do
p = calculos(a,4)
print *, "p=", p
if(p > 0) then
  write (1,*) "p positivo"
else if(p == 0) then
  write (1,*) "p cero"
else
  write (1,*) "p negativo"
end if
close(1)
stop
1 print *, "erro en open"
stop
end program control2


!-------------------
integer function calculos(a,n) result(p)
integer,dimension(n,n), intent(in) :: a
integer,intent(in) :: n
p=0
do i=1,n
  p = p + a(i,i)*a(i,n+1-i)
end do
return
end function calculos
```

Control Fortran

Escribe un programa en Fortran que lea polo teclado o número n de elementos do vector dinámico de enteiros **x** e realice as seguintes operacións:

- Ler do arquivo `secuencia.dat` (que ven ao final do exame) o vector **x**.
- Ler do teclado unha secuencia de enteiros de tres elementos **s**.
- Escribir un subprograma chamado `ocorrencias(...)` (decide o tipo e argumentos) que devolva o número de ocorrencias da secuencia **s** no vector **x**.
- Chamar ao subprograma `ocorrencias` dende o programa principal e garda o número de ocorrencias ao final do arquivo `secuencia.dat`.

EXEMPLO. Proba con $n = 16$, secuencia $\mathbf{s} = (1, 2, 3)$ e co seguinte arquivo `secuencia.dat`:

```
1 2 3 1 2 29 10 0 -3 4 5 18 21 1 2 3
```

On número de ocorrencias de **s** é 2.

SOLUCIÓN:

```
program control3
integer, dimension(:), allocatable :: x
integer, dimension(3) :: s
integer :: ocorrencias
print '("introduce n: ",$)'
read *, n
allocate(x(n))
open(1, file="secuencia.dat", status="old", err=1)
read (1,*) (x(i), i=1,n)
print *, x
print '("introduce a secuencia a buscar: ",$)'
read *, (s(i),i=1,3)
n_ocur = ocorrencias(s, x,n)
write (1,:) n_ocur
close(1)
deallocate(x)
stop
1 print *, "o arquivo secuencia.dat non existe"
stop
end program control3


!-----------------------
integer function ocorrencias(s, x, n) result(n_ocor)
integer, dimension(3), intent(in) :: s
integer, dimension(n), intent(in) :: x
integer, intent(in) :: n
logical :: atopado
n_ocor = 0
do i = 1, n - 2
  atopado = .true.
  do j = 1, 3
    if(s(j) /= x(i + j - 1)) atopado = .false.
  end do
  if(atopado) n_ocor = n_ocor + 1
end do
return
end function ocorrencias
```

Control Fortran 1

Escribe un programa en Fortran que lea por teclado un número enteiro n e reserve memoria dinámicamente para tres vectores $\mathbf{x}, \mathbf{y}, \mathbf{z}$, todos de orde n. O programa debe ler por teclado os vectores $\mathbf{x}$ e $\mathbf{y}$. Logo, o programa principal debe chamar ao subprograma `calcula_z(...)` (debes decidir o seu tipo e argumentos), que calcule o vector $\mathbf{z}$, con elementos $z_i, i = 1, \ldots, n$, dados por:

$$z_i = \begin{cases} x_i \cdot y_i & \exists k \in \{1, \ldots, n\} : x_i = y_k \\ x_i + y_i & \text{noutro caso} \end{cases}$$

Finalmente, o programa principal debe crear o arquivo `vectores.txt` e almacenar nel os vectores $\mathbf{x}, \mathbf{y}, \mathbf{z}$ (cada un nunha fila). Proba con $n = 5, \mathbf{x} = (1, 2, 3, 4, 5), \mathbf{y} = (6, 4, 3, 2, 1)$ e tes que obter $\mathbf{z} = (6, 8, 9, 8, 5)$.

```
program fortran1
real, dimension(:), allocatable :: x, y, z
print*, "Dimension vectores: "
read*, n
allocate(x(n), y(n), z(n))
print*, "Introduce vector v: "
read*, (x(i), i=1,n)
print*, "Introduce vector w: "
read*,(y(i), i=1,n)
 call calcula_z(x,y, z, n)
print*, " Vector z calculado: "
print*, (z(i), i=1,n)
open(2, file="vectores.txt", status="new", err=4)
write(2,*) "Vector x= ", (x(i), i=1,n)
write(2,*) "Vector y= ", (y(i), i=1,n)
write(2,*) "Vector z= ", (z(i), i=1,n)
close(2)
deallocate(x,y,z)
stop
4 print*, "Erro abrindo vectores.txt"
end program fortran1

!*************************************
! Definicion de subprograma
subroutine calcula_z(x, y, z, n)
real, dimension(n), intent(in) :: x, y
integer, intent(in) ::n
real, dimension(n), intent(out):: z
logical:: igual

do i=1,n
   igual=.false.
   do k=1,n
      if (y(k) == x(i)) then
          igual=.true.
          exit
      end if
   end do
   if (igual) then
      z(i)= x(i)*y(i)
   else
      z(i)= x(i) + y(i)
   end if
end do
return
end subroutine calcula_z
```

Control Fortran 2

Escribe un programa en Fortran que lea por teclado un número enteiro n e reserve memoria dinámica para un vector **v** e unha matriz cadrada **a**, ambos de orde n. O programa debe darlle aos elementos do vector **v** os valores $v_i = i^2, i = 1, \ldots, n$. Logo, o programa principal debe chamar ao subprograma `calcula_matriz(...)` (debes decidir o seu tipo e argumentos), que calcule os elementos da matriz **a**, de modo que a_{ij} sexa o número de compoñentes do vector **v** maiores ca $i + j$. Finalmente, o programa principal debe almacenar no arquivo `exame2.dat` a matriz **a** (cada fila nunha liña da terminal). Proba con $n = 5$, debes obter o seguinte arquivo `exame2.dat`:

```
4 4 3 3 3
4 3 3 3 3
3 3 3 3 3
3 3 3 3 2
3 3 3 2 2
```

```
program fortran2
real, dimension(:), allocatable ::v
integer, dimension(:,:), allocatable :: a
print*, "Introduce dimension vector v: "
read*, n
allocate(v(n), a(n,n))
do i=1,n
    v(i)=i**2
end do
 call calcula_matriz(v, n, a)
open(1, file="exame2.dat", status="new", err=4)
do i=1,n
    write(1,*) (a(i,j), j=1,n)
end do
close(1)
deallocate(v, a)
stop
4 print*, "Erro abrindo arquivo exame2.dat"
stop
end program fortran2

!************************************
! Definicion de subprograma calcula_matriz
subroutine calcula_matriz(v, n, a)
integer, intent(in) :: n
real, dimension(n), intent(in) :: v
integer, dimension(n,n), intent(out) :: a
integer :: contador
do i=1,n
  do j=1,n
      contador=0
      ij=i + j
      do k=1,n
          if( v(k) > ij) contador=contador+1
      end do
      a(i,j)=contador
  end do
end do
return
end subroutine calcula_matriz
```

Control Fortran 3

Escribe un programa en Fortran que declare dúas matrices dinámicas **a** e **b** e abra o arquivo `matriz_a.dat`, que debes crear ti co editor gedit e co seguinte contido:

```
5
1 2 3 4 5
6 7 8 9 8
7 6 5 4 3
2 1 2 3 4
6 5 2 3 3
```

O programa debe ler o número da primeira liña e reservar memoria para **a** e **b** de modo que sexan cadradas de orde n. Logo, debe ler as liñas seguintes do arquivo e introducilas na matriz **a**. Despois, debe chamar ao subprograma `calcula_b(...)`, decidindo o seu tipo e argumentos, que calcule os elementos da matriz **b**, definidos por:

$$b_{ij} = \begin{cases} a_{ij} + a_{ji} & \text{se } i \cdot j \text{ é par} \\ a_{ij}^2 + a_{ji}^2 & \text{noutro caso} \end{cases}$$

Para isto debes empregar a función `mod(m,n)`, que calcula o resto da división enteira m/n. Finalmente, o programa principal debe imprimir a matriz **b** por pantalla (cada fila nunha liña da terminal). Tes que obter a seguinte matriz **b**:

```
2 8 58 6 61
8 14 14 10 13
58 14 50 6 13
6 10 6 6 7
61 13 13 7 18
```

```
program fortran3
real, dimension(:,:), allocatable ::a, b
open(2, file="matriz_a.dat", status="old", err=1)
read(2,*) n
allocate(a(n,n), b(n,n))
do i=1,n
    read(2,*) (a(i,j), j=1,n)
end do
close(2)
 call calcula_b(a, n, b)
print*, "Matriz b: "
do i=1,n
    print*, (b(i,j), j=1,n)
end do
deallocate(a,b)
stop
1 print*, "Erro abrindo o arquivo matriz_a.dat"
end program fortran3

!**********************************************
! definicion de subprograma calcula_b
subroutine calcula_b(a, n, b)
integer, intent(in)::n
real, dimension(n,n), intent(in) :: a
real, dimension(n,n), intent(out) :: b

do i=1,n
    do j=1,n
        if (0 == mod(i*j, 2)) then
            b(i,j)=a(i,j) + a(j, i)
        else
```

```
            b(i,j)= a(i,j)**2 + a(j,i)**2
        end if
    end do
end do
return
end subroutine calcula_b
```

Control Fortran 4

Escribe un programa en Fortran que declare un vector **v** e unha matriz cadrada **a**, ambos de orde 5. O programa debe ler por teclado 5 números enteiros entre 1 e 5 (debe comprobar que se atopan nese rango) e os almacene no vector **v**. O programa principal debe chamar ao subprograma `calcula_a(...)` (debes decidir o seu tipo e argumentos), que calcule os elementos da matriz **a** de modo que a_{ij} sexa o n^o de compoñentes de **v** iguais a i ou a j. Finalmente, o programa principal debe almacenar no arquivo `exame4.dat` a matriz **a** (unha fila en cada liña do arquivo) e o vector **v**. Proba coa secuencia de números 0, 3, 1, -3, 7, 5, 2, 4. Debes obter o vector $\mathbf{v} = (3, 1, 5, 2, 4)$ e a seguinte matriz **a**:

```
1 2 2 2 2
2 1 2 2 2
2 2 1 2 2
2 2 2 1 2
2 2 2 2 1
```

```
program fortran4
integer, dimension(5) :: v
integer, dimension(5,5) :: a
integer:: contador
n=5; i=1
print*, "Introduce numeros enteiros entre 1 e 5: "
do
  read*, m
  if (m > 0 .and. m < 6) then
      v(i)=m; i=i+1
  end if
  if (i > n) exit
end do
print*,(v(i), i=1,n)
 call calcula_a(v, n, a)
open(1, file="exame4.dat", status="new", err=1)
do i=1,n
    write(1,*) (a(i,j), j=1,n)
end do
close(1)
stop
1 print *, "erro: o arquivo exame4.dat xa existe"
stop
end program fortran4

!---------------------------------------
subroutine calcula_a(v,n, a)
integer, intent(in)::n
integer, dimension(n),intent(in) ::v
integer, dimension(n,n), intent(out) :: a
do i=1,n
    do j=1, n
        l=0
        do k=1,n
            if (v(k) == i .or. v(k) == j) l=l + 1
        end do
        a(i,j)=l
    end do
end do
return
end subroutine calcula_a
```

Control Fortran 1

Escribe co editor un arquivo chamado `exame1.dat` co seguinte contido:

```
10
1 2 3 4 5 6 7 8 9 10
```

Escribe un programa que lea dende este arquivo o número da primeira fila á variábel n e reserve dinámicamente un vector **v** de n compoñentes. Logo, debe ler dende o arquivo e almacenar en **v** os valores da segunda liña do arquivo. Entón o programa principal debe chamar ao subprograma `modifica(...)`, que lea números por teclado até que o usuario introduza un cero, e calcule a suma s destos números. Nese momento, o subprograma debe modificar o vector **v** sumando s a cada compoñente de **v**. Finalmente, o programa principal debe mostrar por pantalla o novo vector **v**.

SOLUCIÓN:

```fortran
program exame1
real,dimension(:),allocatable :: v
open(1,file="exame1.dat",status="old",err=1)
read (1,*) n
allocate(v(n))
read (1,*) (v(i),i=1,n)
close(1)
call modifica(v,n)
print *, "v=", v
deallocate(v)
stop
1 print *, "erro abrindo exame1.dat"
stop
end program exame1

!-----------------------------
subroutine modifica(v,n)
real,dimension(n),intent(inout) :: v
integer,intent(in) :: n
s = 0
print *, "introduce numeros (0 para rematar)"
do
  read *, x
  s = s + x
  if(0==x) exit
end do
do i=1,n
  v(i)=v(i)+s
end do
return
end subroutine modifica
```

Control Fortran 2

Escribe un programa que lea por teclado un número enteiro n, e calcule os valores $x_i = 2^{-i}, i = 0, \ldots, n$. Para cada x_i debe chamar ao subprograma `serie(...)` que debe calcular o valor aproximado da seguinte serie:

$$y_i = \sum_{j=0}^{\infty} \frac{x_i^j}{j^2+1}, \qquad i = 0, \ldots, n \tag{1}$$

tendo en conta só os sumandos con valor absoluto maior ca 10^{-5}. Finalmente, o programa principal debe crear o arquivo `exame2.dat` e almacenar nel os valores $y_i, i = 0, \ldots, n$, cada un nunha liña distinta.
NOTA: Proba con $n = 3$, de modo que $x_0 = 1$, $x_1 = 0.5$, $x_2 = 0.25$, $x_3 = 0.125$ e debes obter $y_0 = 2.073$, $y_1 = 1.318$, $y_2 = 1.139$, $y_3 = 1.065$.

SOLUCIÓN:

```
program exame2
print *, "introduce n:"
read *, n
open(1,file="exame2.dat",status="new",err=1)
do i = 0, n
  x = 1./2**i; y = serie(x)
  write (1,*) y
end do
close(1)
stop
1 print *, "erro en open"
stop
end program exame2

!-------------------------
function serie(x) result(y)
real,intent(in) :: x
y=0; j=0
do
  sumando = x**j/(j**2 + 1)
  if(sumando < 1e-5) exit
  y = y + sumando; j = j + 1
end do
return
end function serie
```

Control Fortran 3

Crea co editor un arquivo chamado `exame3.dat` co seguinte contido:

```
3
6 2 3
9 5 1
7 8 4
```

Escribe un programa que lea o número que se atopa na primeira liña do arquivo `exame3.dat` á variábel n e reserve memoria para unha matriz cadrada de orde n con valores enteiros, que debe ler dende as liñas 2-4 do anterior arquivo. Logo, o programa principal debe chamar ao subprograma `procesa(...)`, que debe calcular a suma dos elementos pares e a suma dos elementos impares da matriz. O programa principal debe mostrar por pantalla estos dous valores calculados polo subprograma.
NOTA: Usando o arquivo `exame3.dat`, o programa debe obter 20 para a suma dos elementos pares e 25 para a suma dos impares.

SOLUCIÓN:

```fortran
program exame3
integer,dimension(:,:),allocatable :: a
integer :: sp, si

open(1,file="exame3.dat",status="old",err=1)
read (1,*) n
allocate(a(n,n))
do i=1,n
  read (1,*) (a(i,j),j=1,n)
end do
close(1)

call procesa(a,n,sp,si)

print *, "sp=", sp, "si=", si

deallocate(a)
stop
1 print *, "erro en open"
stop
end program exame3


!_____________________________
subroutine procesa(a,n,sp,si)
integer,dimension(n,n),intent(inout) :: a
integer,intent(out) :: sp,si
integer,intent(in) :: n
np=0;sp=0;ni=0;si=0
do i=1,n
  do j=1,n
    if(mod(a(i,j),2)==0) then
      sp=sp+a(i,j)
    else
      si=si+a(i,j)
    end if
  end do
end do
return
end subroutine procesa
```

Control Fortran 4

Crea co editor de texto o arquivo `exame4.dat` co seguinte contido:

```
4
4 2 1 3
2 3 2 1
3 1 2 4
2 2 1 3
```

Escribe un programa que lea o número n dende a primeira liña deste arquivo e reserve memoria para unha matriz **a** cadrada de orde n con elementos enteiros. O programa debe chamar ao subprograma `suma(...)`, que debe percorrer a matriz e sumar os seus elementos detendo o percorrido cando esta suma supere o valor 10. Finalmente, o programa principal debe mostrar por pantalla o valor da suma.
NOTA: Usando o arquivo `exame4.dat` anterior, debes obter unha suma=12.

SOLUCIÓN:

```fortran
program exame4
integer,dimension(:,:),allocatable :: a
integer :: suma

open(1,file="exame4.dat",status="old",err=1)
read (1,*) n
allocate(a(n,n))
do i=1,n
  read (1,*) (a(i,j),j=1,n)
end do
close(1)

m = suma(a,n)
print *, "suma=", m

deallocate(a)
stop
1 print *, "erro en open"
stop
end program exame4


!-----------------------
integer function suma(a,n)
integer,dimension(n,n),intent(in) :: a
integer,intent(in) :: n

suma = 0
filas: do i=1,n
  do j=1,n
    suma = suma + a(i,j)
    print *, i, j, a(i,j), suma
    if(suma > 10) exit filas
  end do
end do filas

return
end function suma
```

Control de programación en Fortran

Escribe un programa en fortran que lea do arquivo `matriz.dat` unha matriz cadrada a de orde n. Define un subprograma chamado `calculos()` (debes decidir o tipo e argumentos que necesita) que calcule a suma dos elementos pares de a e conte o número de elementos impares en a. O programa principal ten que chamar ó subprograma e visualizar os resultados na pantalla: a matriz a (unha fila en cada liña), a suma dos elementos pares e o número de elementos impares.

Por exemplo, se o arquivo `matriz.dat` contén:

```
4
1 2 4 6
4 3 7 5
4 8 0 5
6 4 7 1
```

onde a primeira liña do arquivo é a orde da matriz, o resultado sería a suma 38 e o número de elementos impares 7.

```fortran
program control1
integer, dimension(:,:), allocatable ::a
open(1, file="matriz.dat",status="old", err=4)
read(1,*) n
allocate(a(n,n))
print*, "Matriz a: "
do i=1,n
  read(1,*) (a(i,j), j=1,n)
  print*, (a(i,j), j=1,n)
end do
close(1)
call calculos(a, n, nsuma, nimpares)
print*, "Suma pares = ", nsuma
print*, "Numero de impares= ", nimpares
deallocate(a)
stop
4 print*, "Erro abrindo matriz.dat"
end program control1

!*********************************
! Definicion de subprograma
subroutine calculos(b, m, suma, impares)
integer, dimension(m,m), intent(in) :: b
integer, intent(in) :: m
integer, intent(out) :: impares, suma
suma = 0
impares = 0
do i=1,m
  do j=1,m
    if(mod(b(i,j), 2) == 0) then
      suma = suma + b(i,j)
    else
      impares = impares + 1
    end if
  end do
end do
return
end subroutine calculos
```

Control de programación en Fortran

Escribe un subprograma `polinomio()` (debes decidir o tipo e argumentos) que calcule o n-ésimo polinomio de Tchevyshev para un valor de x, definido por:

$$P_n(x) = \prod_{k=1}^{n} (kx - 2) \tag{1}$$

Escribe un programa principal que pida ó usuario a orde do polinomio n e chame ao subprograma `polinomio()` para que calcule o n-ésimo polinomio para valores de x=0, 0.1, 0.2, 0.3,0.4,0.5,... mentres que o valor do polinomio sexa positivo. O programa ten que avaliar o número de polinomios calculados, se este número e maior que 10 mostrar a mensaxe "Menos de 10 polinomios" e en caso contrario mostrar a mensaxe "Máis de 10 polinomios". O programa principal ten que gardar nun arquivo chamado `control.dat` o valor de x e o do polinomio (un en cada liña). Por exemplo, se consideras $n = 7$ o arquivo `control.dat` debe conter:

```
0.00000000      -128.000000
0.100000001      -25.3955517
0.200000003      -2.32243180
```

```
program control2
integer :: contador
print*, "Introduce n: "
read*, n
open(2, file="control.dat", status="new", err=5)
x=0; h=0.1; contador = 0
do
  pn=polinomio(n,x)
  if (pn > 0) exit
  write(2,*) x, pn
  contador = contador + 1
  x = x+h
end do
close(2)
if(contador < 10) then
  print*, "Menos de 10 polinomios"
else
  print*, "Mais de 10 polinomios"
end if
stop
5 print*, "Erro abrindo control.dat"
end program control2
!****************************************
! Definicion de subprograma
function polinomio(n, x)
real, intent(in):: x
integer, intent(in) :: n
  pnx = 1
  do k = 1, n
      pnx = pnx*(k*x-2)
  end do
  polinomio = pnx
return
end function polinomio
```

Control de programación en Fortran

Escribe un programa que lea por teclado repetidamente números enteiros ata que o número de elementos pares e impares sexa igual. Sexa n o número de elementos pares introducidos, tes que definir un subprograma chamado `xera_matriz()` (debes decidir o tipo e argumentos) que xere unha matriz b de orde $n \times 2n$, onde cada elemento $b_{ij} = ij$, $i = 1, \ldots, n$, $j = 1, \ldots, 2n$.

Por exemplo, se introduces os números 2,4,5,6,8,3,1,7, o programa debe rematar de ler números logo de introducir o 7. Nese caso $n = 4$, e o arquivo `matrizb.dat` contén:

```
1          2          3          4          5          6          7          8
2          4          6          8         10         12         14         16
3          6          9         12         15         18         21         24
4          8         12         16         20         24         28         32
```

```fortran
program control3
integer, dimension(:,:), allocatable :: b
npar=0; nimpar=0
do
  print*, "Introduce numero: "
  read*, n
  if (mod(n,2) == 0) then
    npar = npar + 1
  else
    nimpar = nimpar + 1
  end if
  if (npar == nimpar) exit
end do
n=npar
allocate(b(n,2*n))
call xera_matriz(b, n)
open(3,file="matrizb.dat", status="new", err=6)
n2=2*n
do i=1,n
  write(3,*) (b(i,j), j=1,n2)
end do
close(3)
deallocate(b)
stop
6 print*, "Erro abrindo matrizb.dat"
end program control3
!*********************************
subroutine xera_matriz(a, m)
integer, dimension(m,m), intent(out) :: a
integer, intent(in)::m
m2=2*m
do i=1,m
  do j=1,m2
    a(i,j)=i*j
  end do
end do
return
end subroutine xera_matriz
```

Control de programación en Fortran

Define un subprograma chamado `cifras` (debes decidir o tipo e argumentos) que descompoña un número enteiro nas súas cifras. O programa principal ten que ler por teclado dous números x e y e almacenar no arquivo `cifras.dat` as cifras do número x (todas nunha única liña), as cifras do número y e noutra liñha as cifras comúns a ambos números. Usa vectores estáticos de 50 elementos. Por exemplo, se introduces os números $x = 123456$ e $y = 13682$, o arquivo `cifras.dat` contén:

```
Cifras x: 6 5 4 3 2 1
Cifras y: 2 8 6 3 1
Cifras comuns: 6 3 2 1
```

```fortran
program control4
integer, dimension(50) :: xc, yc, cc
integer :: x, y
print*, "Introduce x e y: "
read*, x, y
call cifras(xc, 50, x, nxc)
call cifras(yc, 50, y, nyc)
open(2, file="cifras.dat", status="new", err=7)
write(2,*) "Cifras x: ", (xc(i), i=1,nxc)
write(2,*) "Cifras y: ", (yc(i), i=1,nyc)
ncc=0
do i=1, nxc
  do j=1,nyc
    if( xc(i) == yc(j)) then
        ncc = ncc + 1
        cc(ncc) = xc(i)
        exit
    end if
  end do
end do
write(2,*) "Cifras comuns: ", (cc(i), i=1,ncc)
close(2)
stop
7 print*, "Erro abrindo arquivo cifras.dat"
end program control4
!****************************
subroutine cifras(v, n, x, nc)
integer, intent(in) ::x, n
integer, dimension(n), intent(out) :: v
integer, intent(out) :: nc
m=x
nc=1
do
  v(nc)=mod(m,10)
  m=m/10
  if (m == 0) exit
  nc = nc + 1
end do
print*, "Cifras de ", x , " son: ", (v(i), i=1,nc)
return
end subroutine cifras
```

Control de programación en Fortran

Escribe un programa que lea dos vectores **v** e **w** de dimensión n dende o arquivo `vectores.dat`. Define un subprograma chamado `construir_matriz()` (debes decidir o tipo e argumentos) que calcule unha matriz **a** de orde $n \times n$, onde cada elemento da matriz a_{ij} ven dado pola expresión:

$$a_{ij} = \begin{cases} \sum_{k=1}^{i} v_k^2 & j \leq i \\ w_j v_i & j > i \end{cases}$$

O programa principal ten que chamar ao subprograma `construir_matriz()` para calcular a matriz **a** e visualizala na pantalla (unha fila en cada liña). Por exemplo, se o arquivo `vectores.dat` conten os seguintes vectores:

```
5
2 3 4 5 6
2 1 5 4 3.1
```

onde a primeira liña é a dimensión dos vectores, a segunda o vector **v** e a terceira o vector **w**, a matriz calculada **a** sería:

```
4.00000000      2.00000000      10.0000000      8.00000000      6.19999981
13.0000000      13.0000000      15.0000000      12.0000000      9.29999924
29.0000000      29.0000000      29.0000000      16.0000000      12.3999996
54.0000000      54.0000000      54.0000000      54.0000000      15.5000000
90.0000000      90.0000000      90.0000000      90.0000000      90.0000000
```

```fortran
program control5
real, dimension(:),allocatable ::v, w
real, dimension(:,:), allocatable ::a
open(2, file="vectores.dat", status="old", err=5)
read(2, *) n
allocate(v(n), w(n), a(n,n))
read(2,*)(v(i), i=1,n)
read(2,*)(w(i), i=1,n)
call construir_matriz(v,w, a, n)
print*, "Matriz a: "
do i=1,n
    print*, (a(i,j), j=1,n)
end do
deallocate(v,w,a)
stop
5 print*, "Erro abrindo vectores.dat"
end program control5
!****************************************
! Definicion de subprograma
subroutine construir_matriz(v, w, b, m)
real, dimension(m), intent(in) :: v, w
real, dimension(m,m), intent(out) :: b
integer, intent(in)::m
do i=1,m
  do j=1,m
    if (j <= i) then
        b(i,j)=0
        do k=1,i
            b(i,j)= b(i,j) + v(k)*v(k)
```

```
        end do
    else
        b(i,j)=w(j)*v(i)
    end if
  end do
end do
return
end subroutine construir_matriz
```

Control de Fortran

Crea o arquivo `datos1.dat` coas dúas columnas de números enteiros:

```
8 8
4 3
5 5
3 3
4 4
7 5
7 7
8 5
9 1
6 6
1 4
```

Escribe un programa en Fortran que defina un subprograma chamado `contarCoincidencias()`, co tipo e argumentos axeitado, que lea os datos do arquivo `datos1.dat` liña a liña. Ademáis, debe sumar os valores de todas as liñas nas que coinciden os seus números ata que a suma chegue a 50. O subprograma ten que devolver o número de liñas sumadas. Sexa n o número de liñas sumadas.

O programa principal debe chamar o subprograma `contarCoincidencias()`, e crear un vector **v** de dimensión n, onde cada elemento v_i, con $i = 1, \ldots, n$, é:

$$v_i = e^{2i} \operatorname{sen}\left(\frac{2\pi i}{3}\right) \tag{1}$$

Posteriormente, o programa principal debe mostrar por pantalla o vector **v** nunha única liña con 2 cifras decimais. NOTA: debes obter $n = 5$ e o vector `v= 6.40 -47.28 -0.00 2581.59 -19075.38`.

```fortran
program control1
integer :: contarCoincidencias
real, dimension(:), allocatable::v
real, parameter :: pi = 3.14159
n=contarCoincidencias()
print*, "Numero de linhas: ", n
allocate(v(n))
do i=1,n
    v(i)=exp(real(2*i))*sin(2*pi*i/3)
end do
print '(a$)', "v="
do i=1,n
    print '(f10.2 $)', v(i)
end do
deallocate(v)
stop
end program control1

integer function contarCoincidencias()
integer :: suma
nlinhas=0
open(1, file="datos1.dat", status="old", err=3)
do
    read(1,*, end=4) n, m
    if (n == m) then
        suma = suma + n + m
        nlinhas = nlinhas + 1
        if (suma >= 50) exit
```

```
    end if
end do
4 close(1)
contarCoincidencias = nlinhas
return
3 print*, "Erro abrindo datos1.dat"
end function contarCoincidencias
```

Control de Fortran

Crea o arquivo `datos2.dat` que almacena unha matriz de números enteiros de orde n, onde a primeira liña contén a orde da matriz:

```
5
1 3 2 5 4
6 4 3 2 4
5 4 3 6 1
4 8 9 6 4
5 3 4 1 2
```

Escribe un programa que lea o arquivo `datos2.dat` e almacene os datos na matriz **a**. Define un subprograma chamado `calculaVector()`, cos argumentos e tipo axeitado, que calcule un vector **v** de lonxitude n (a orde da matriz) cos elementos v_i dados por:

$$v_i = \sum_{j=1}^{i} \sum_{k=1}^{i} a_{jk} \quad 1 \leq i \leq n \tag{2}$$

O programa principal ten que chamar ó subprograma `calculaVector()` para calcular o vector **v** e visualizalo na pantalla. Posteriormente, debe visualizar por pantalla os elementos de **v** mentres que a suma sexa menor que 50. NOTA: debes obter o vector **v**=`1 14 31 71 99` e posteriormente, ten que mostrar os valores `1 14 31`.

```fortran
program control2
integer, dimension(:,:), allocatable ::a
integer, dimension(:), allocatable:: v
integer:: suma
open(2,file="datos2.dat", status="old", err=3)
read(2,*) n
allocate(a(n,n), v(n))
do i=1,n
    read(2,*) (a(i,j), j=1,n)
end do
call calculaVector(a, n, v)
print*, "Vector v= ", (v(i), i=1,n)
print '(a,$)', "Valores de v: "
suma=0
i=1
do
    suma= suma + v(i)
    if (suma < 50) then
        print '(i4,$)', v(i)
        i=i+1
    else
        exit
    end if
end do
deallocate(a, v)
stop
3 print*, "Erro abrindo arquivo datos2.dat"
end program control2

subroutine calculaVector(a, n, x)
integer, dimension(n,n), intent(in)::a
integer, intent(in):: n
```

```
integer, dimension(n), intent(out)::x
do i=1,n
    x(i)=0
    do j=1,i
        do k=1,i
            x(i) = x(i) + a(j,k)
        end do
    end do
end do
return
end subroutine calculaVector
```

Control de Fortran

Escribe un programa en Fortran que declare un vector estático de números enteiros **v** con 100 elementos. O programa ten que ler números por teclado ata que a súa suma sexa maior que 30 e almacenalos no vector **v**. Escribe un subprograma chamado `calculaVector()`, cos argumentos e tipo axeitado, que devolva un vector **y** cos valores de **v** no intervalo [5, 10]. O programa principal ten que chamar ó subprograma `calculaVector()` e, posteriormente, gardar no arquivo `exame3.dat` os vectores **v** e **y** (un vector en cada liña).

NOTA: se introduces 3,2,7,1,6,5,10,9,7 o contido do arquivo `exame3.dat` sería:

```
v= 3 2 7 1 6 5 10
y= 7 6 5 10
```

```fortran
program control3
integer,dimension(100)::v, y
open(1,file='exame3.dat',status='new',err=1)
n=0
suma=0
do
    print '(a,$)', "No: "
    read*, v(n)
    suma = suma + v(n)
    n=n+1
    if (suma  > 30) exit
end do
write (1,*) "vector v: ", (v(i), i=1,n)
call calculaVector(v, n, y, ny)
write (1,*) "vector y: ", (y(i), i=1,ny)
close(1)
stop
1 print *,'erro en open abrindo exame3.dat'
stop
end program control3

subroutine calculaVector(v,n,y, ny)
integer, dimension(100), intent(in)::v
integer, intent(in)::n
integer, dimension(100),intent(out)::y
integer, intent(out)::ny
ny=0
do i=1,n
    if(v(i) >=5 .and. v(i) <= 10) then
        ny=ny+1
        y(ny)=v(i)
    end if
end do
return
end subroutine calculaVector
```

Control de Fortran

Crea co editor de texto un arquivo chamado `datos5.dat` co seguinte contido:

```
8
3 1 0 8 2 7 9 5
```

onde a primeira liña é o número n de elementos na segunda liña. Escribe un programa en Fortran que lea o arquivo `datos5.dat` e o almacene no vector **v** de dimensión n. Define un subprograma chamado `convirte()`, cos argumentos e tipo axeitado, que calcule un vector **w**, da mesma lonxitude n que **v**, de modo que o elemento w_i, con $1 \leq i \leq n$, sexa: se i é múltiplo de 3, entón $w_i = \sum_{j=1}^{i} j^2 v_j$; en caso contrario, $w_i = i^2$.

O programa principal debe chamar ó subprograma `convirte()`, e logo visualizar na terminal o vector **w**, informando se o vector **w** ten algún elemento repetido ou non (non mostres os elementos repetidos).

NOTA: debes obter **w**=1,4,7,16,25,437,49,64, que non ten elementos repetidos.

```fortran
program control
integer,dimension(:), allocatable::v, w
logical :: repetido=.false.
open(7, file="datos5.dat", status="old", err=2)
read(7,*) n
allocate(v(n), w(n))
read(7,*)(v(i), i=1,n)
close(7)
print*, "vector v: ", (v(i), i=1,n)
call convertir(v, n, w)
print*, "vector w: ", (w(i), i=1,n)
filas: do i=1,n
    k=w(i)
    do j=i+1,n
        if(w(j)==k) then
            repetido=.false.
            exit filas
        end if
    end do
end do filas
if(repetido) then
    print *,'w ten elementos repetidos'
else
    print *,'w non ten elementos repetidos'
end if
deallocate(v,w)
stop
2 print*, "Erro abrindo o arquivo datos5.dat"
end program control

subroutine convertir(x,n,y)
integer, dimension(n), intent(in)::x
integer, intent(in)::n
integer, dimension(n),intent(out)::y
do i=1,n
    if(0==mod(i,3)) then
        s=0
        do j=1,i
            s=s+j*j*x(j)
```

```
        end do
        y(i)=s
    else
        y(i)=i*i
    end if
end do
return
end subroutine convertir
```

Control de Fortran

Co editor Kate, crea un arquivo chamado `entrada.txt` co seguinte contido:

```
3
1 2 3
4 5 6
7 8 9
```

Escribe un programa en Fortran que lea o primeiro número do arquivo anterior á variábel n (neste caso n=3), reserve unha matriz enteira cadrada **a** de orde n e lea dende o arquivo anterior os seus elementos. Logo, o programa debe chamar a un subprograma `transforma(...)`, do tipo e cos argumentos axeitados, que transforme a matriz **a** a outra matriz **b** de modo que $b_{ij} = a_{n-j+1,n-i+1}$, con $i, j = 1, \ldots, n$. Finalmente, o programa principal debe mostrar por pantalla os elementos da matriz **b** mentres que a súa suma sexa inferior a 20. Usando o arquivo `entrada.txt` anterior, o programa debe mostrar por pantalla: 9, 6, 3.

SOLUCIÓN:

```fortran
program exame
integer,dimension(:,:),allocatable :: a,b
open(1,file='entrada.txt',status='old',err=1)
read (1,*) n
allocate(a(n,n),b(n,n))
do i=1,n
  read (1,*) (a(i,j),j=1,n)
end do
close(1)
call transforma(a,n,b)
suma=0
filas: do i=1,n
  do j=1,n
    suma=suma+b(i,j)
    if(suma>20) exit filas
    print *,b(i,j)
  end do
end do filas
print *,'b='
do i=1,n
  print *,(b(i,j),j=1,n)
end do
deallocate(a,b)
stop
1 print *,'erro: entrada.txt non existe'
stop
end program exame

subroutine transforma(a,n,b)
integer,dimension(n,n),intent(in) :: a
integer,intent(in) :: n
integer,dimension(n,n),intent(out) :: b
do i=1,n
  do j=1,n
    b(i,j)=a(n-j+1,n-i+1)
  end do
end do
return
end subroutine transforma
```

Control de Fortran

Escribe un programa en Fortran que lea por teclado valores enteiros positivos, almacenando estos valores nun vector **v** estático de lonxitude 10, e remate cando o usuario introduza un valor negativo ou cando se introduza o décimo valor. Sexa n o número de valores introducidos, sen contar o valor -1 (en caso de que se tecleara). O programa debe crear unha matriz enteira cadrada **a** de orde n e chamar a un subprograma `matriz(...)`, co tipo e argumentos axeitados. Este subprograma debe calcular os elementos a_{ij} da matriz **a**, dados por:

$$a_{ij} = \sum_{k=1}^{n} v_{\text{mín}(i+k,n)} v_{\text{máx}(j-k,1)}, \quad i,j = 1,\ldots,n \tag{1}$$

onde: $\text{mín}(x,y) = x$ se $x \leq y$ e $\text{mín}(x,y) = y$ en caso contrario; $\text{máx}(x,y) = x$ se $x \geq y$ e $\text{máx}(x,y) = y$ en caso contrario. Finalmente, o programa principal debe almacenar os elementos da matriz **a** no arquivo `saida.txt`, nunha única liña. Antes de cada execución (agás a primeira), on esquezas borrar, co comando `rm -f saida.txt`, o arquivo `saida.txt`. Introduce os números $1, 2, 3, -1$ e tes que obter o arquivo `saida.txt` seguinte:

```
8  9  9  8  9  9 10 12 12
```

Control de Fortran

Escribe un programa en Fortran chamado `exame.f90` que lea un número enteiro n e logo n valores enteiros, que debes almacenar nun vector **v** de lonxitude n. O programa debe crear un arquivo novo chamado `saida.txt`, e chamar repetidamente a un subprograma `reduce(...)`, do tipo e cos argumentos axeitados, que transforme o vector **v** de modo que:

$$v_i = \begin{cases} v_i + v_{n-i+1} & 1 \leq i \leq m \\ v_{i-m} + v_i & m < i \leq n \end{cases} \tag{1}$$

onde $m = \lfloor n/2 \rfloor$. A chamada ao subprograma repetirase ata que a suma dos elementos do vector **v** sexa maior que 100. En cada iteración, o programa debe engadir o vector **v** calculado ao final do arquivo `saida.txt` (o vector **v** debe ir nunha liña distinta en cada iteración). Proba con $n = 5$ e **v**=(1,2,3,4,5). Antes de cada execución (agás a primeira), on esquezas borrar o arquivo `saida.txt` co comando `rm -f saida.txt`. Tes que obter o seguinte arquivo `saida.txt`.

```
6 6 9 10 14
20 16 29 26 43
```

SOLUCIÓN:

```fortran
program exame
integer,dimension(:),allocatable :: v
print '("n? ",$)';read *,n
allocate(v(n));m=n/2
print '("v? ",$)';read *,v
open(1,file='saida.txt',status='new',err=1)
do
  call reduce(v,m,n)
  write (1,*) v
  if(sum(v)>100) exit
end do
close(1)
deallocate(v)
stop
1 print *,'erro: saida.txt xa existe'
stop
end program exame


subroutine reduce(v,m,n)
integer,dimension(n),intent(inout) :: v
integer,intent(in) :: n,m
do i=1,m
  v(i)=v(i)+v(n-i+1)
end do
do i=m+1,n
  v(i)=v(i-m)+v(i)
end do
return
end subroutine reduce
```

Control de Fortran

Co editor Kate crea un arquivo chamado `entrada.txt` co seguinte contido:

```
1 2 3 4 5
9 4 2
1 2 7
```

Escribe un programa en Fortran que lea por teclado a primeira liña deste arquivo a un vector estático **x**, e as dúas liñas restantes a unha matriz **a**, tamén estática, de orde 2×3. O programa debe chamar a un subprograma `porcentaxe(...)`, que sume os elementos do vector **x** ata que a súa suma sexa superior a 7. Chamando m ao número de elementos sumados, o subprograma debe calcular a porcentaxe de elementos da matriz **a** maiores que m. Finalmente, o programa principal debe mostrar por pantalla a devandita porcentaxe con 6 díxitos e 2 cifras decimais. Co arquivo `entrada.txt` anterior, debes obter unha porcentaxe do 33.33 %.

SOLUCIÓN:

```fortran
program exame
integer,dimension(5) :: x
integer,dimension(2,3) :: a
open(1,file='entrada.txt',status='old',err=1)
read (1,*) x
do i=1,2
  read (1,*) (a(i,j),j=1,3)
end do
close(1)
p=porcentaxe(x,5,a,2,3)
print '("porcentaxe=",f6.2,"%")',p
stop
1 print *,'erro: o arquivo entrada.txt non existe'
stop
end program exame


function porcentaxe(x,n,a,nf,nc)
integer,dimension(n),intent(in) :: x
integer,dimension(nf,nc),intent(in) :: a
integer,intent(in) :: n,nf,nc
s=0;porcentaxe=0
do m=1,5
  s=s+x(m)
  if(s>7) exit
end do
do i=1,nf
  do j=1,nc
    if(a(i,j)>m) porcentaxe=porcentaxe+1
  end do
end do
porcentaxe=100.*porcentaxe/nf/nc
return
end function porcentaxe
```

Control de Fortran

Escribe un programa en Fortran chamado `exame.f90` que lea por teclado o grao n e os coeficientes a_i, con $i = 0, \ldots, n$, dun polinomio $p(x) = \sum_{i=0}^{n} a_i x^i$. O programa debe chamar a un subprograma `integral_polinomio(...)`, do tipo e cos argumentos axeitados, que calcule os coeficientes b_i, con $i = 1, \ldots, n+1$, do polinomio $q(x) = \sum_{i=1}^{n+1} b_i x^i$ resultante de integrar $p(x)$, é decir, $q(x) = \int p(x)dx$. Estos coeficientes están dados por $b_i = \frac{a_{i-1}}{i}$. Finalmente, o programa principal debe almacenar no arquivo `saida.txt` os vectores **a** e **b** cos coeficientes dos dous polinomios $p(x)$ e $q(x)$, un en cada liña, con formato real de ancho 6 e 2 decimais. Agás na primeira execución, lembra borrar o arquivo `saida.txt` co comando `rm -f saida.txt`. Usando $n = 3$ e o polinomio $p(x) = 4 + 3x + 2x^2 + x^3$, de modo que $\mathbf{a} = (4, 3, 2, 1)$, tes que obter o arquivo `saida.txt` seguinte:

```
4.00  3.00  2.00  1.00
4.00  1.50  0.67  0.25
```

SOLUCIÓN:

```fortran
program exame
real,dimension(:),allocatable :: a,b
print '("n? ",$)';read *,n
allocate(a(0:n),b(n+1))
print '("a? ",$)';read *,a
call integral_polinomio(a,n,b)
open(1,file='saida.txt',status='new',err=1)
do i=0,n
  write (1,'(f6.2,$)') a(i)
end do
write (1,*) ''
do i=1,n+1
  write (1,'(f6.2,$)') b(i)
end do
close(1)
deallocate(a,b)
stop
1 print *,'erro: arquivo saida.txt xa existe'
stop
end program exame


subroutine integral_polinomio(a,n,b)
real,dimension(0:n),intent(in) :: a
integer,intent(in) :: n
real,dimension(n+1),intent(out) :: b
do i=1,n+1
  b(i)=a(i-1)/i
end do
return
end subroutine integral_polinomio
```

Control de Fortran

Escribe un programa en fortran chamado `exame1.f90` que lea por teclado un número enteiro positivo n (usa $n = 5$). Crea un vector **p** de dimensión n, onde cada elemento do vector p_i, $i = 1, \ldots, n$ calcúlase co subprograma `calculaValor()` (que debes definir cos argumentos axeitados) e que se define como:

$$p_i = \sum_{n=0}^{i} n \cos\left(\frac{n\pi}{10}\right)$$

No programa principal,calcula unha matriz **a** cadrada de orde n, onde cada elemento a_{ij} sexa o número de elementos do vector **p** (comezando polo índice 1 e volvendo a comezar polo índice 1 cando se chegue ó final do vector) para que se cumpra a ecuación:

$$\sum_k p_k < ij^2$$

Garda no arquivo `resultado.txt` a matriz **a**. NOTA: para $n = 5$ o vector **p** sería:

```
p=   0.951056600        2.56909132        4.33244896        5.56852102        5.56852722
```

SOLUCIÓN:

```fortran
program exame1
real,dimension(:,:), allocatable :: a
real,dimension(:), allocatable :: p
print '("n= ", $)'
read*, n
allocate(p(n), a(n,n))
do i=1,n
    p(i)=calculaValor(i)
end do
print*, "p= ", p
do i=1,n
    do j=1,n
        numbral=i*j*j
        nv=0
        suma=0
        k=1
        do
            suma= suma + p(k)
            k=k+1
            if(k>n) k=1
            nv= nv +1
            if (suma > numbral) exit
        end do
        a(i,j)=nv
    end do
end do
open(1,file="resultados.txt", status="new", err=2)
do i=1,n
    write(1, *)(a(i,j), j=1,n)
end do
close(1)
stop
2 print*, "Erro escribindo en resultados.txt"
end program exame1

!*********************************************
function calculaValor(m)
integer, intent(in) :: m
```

```
real ,parameter :: pi=3.14159
f(n)=n*cos(pi*n/10)
suma= 0
do i=0, m
    suma = suma + f(i)
end do
calculaValor=suma
return
end function calculaValor
```

Control de Fortran

Escribe un programa en fortran chamado `exame2.f90` que lea por teclado un número enteiro positivo n maior que 3. O programa non debe rematar mentres o usuario introduza un número incorrecto. Crea un vector **p** de dimensión n, onde cada elemento p_i, $i = 1, \ldots, n$ ven dado pola expresión:

$$p_i = \sum_{j=0}^{k-1} \frac{5j}{k^2} \cos\left(\frac{i\pi j}{20k}\right) \qquad k = 100$$

Define un subprograma chamado **calculos()**, decide os argumentos axeitados, que que calcule un vector **y** dado por: $y_1 = 1$ e $y_i = p_i - p_{i-1}$, $i = 2, \ldots, n$.

Desde o programa principal chama ó subprograma `calculos()` e garda no arquivo `resultados.txt` os vectores **p** (co formato por defecto) e **y** (como reais con expoñente).
NOTA:con $n = 5$ o contido do arquivo `resultados.txt` sería:

```
p=    2.45990586       2.41486955       2.34062266       2.23836994       2.10976601
y=     0.10E+01   -0.45E-01   -0.74E-01   -0.10E+00   -0.13E+00
```

SOLUCIÓN:

```fortran
program exame1
real,dimension(:), allocatable :: p, y
real, parameter :: pi=3.14159
do
    print '("n= ", $)'
    read*, n
    if (n > 3) exit
end do
allocate(p(n), y(n))
k=100
do i=1,n
    t=0
    do j=0, k-1
        t= t + 5*j*cos(i*pi*j/20/k)/k**2
    end do
    p(i)= t
end do
call calculos(p, y, n)
open(1, file="resultados.txt", status="new", err=4)
write(1, '("p= ", $)')
write(1, *) (p(i), i=1,n)
write(1, '("y= ", $)')
do i=1,n
    write(1, '(e12.2, t20, $)') y(i)
end do
close(1)
deallocate(y,p)
stop
4 print*, "Erro abrindo resultados.txt"
end program exame1

!*************************************************
subroutine calculos(x, z, n)
real, dimension(n), intent(in) :: x
real, dimension(n), intent(out):: z
integer, intent(in) :: n
z(1)=1
do i=2,n
    z(i)=x(i)-x(i-1)
end do
```

```
return
end subroutine calculos
```

Control de Fortran

Co editor de texto crea o arquivo `datos3.txt` con un número enteiro en cada liña. Por exemplo cos números $2, 3, 1, 1, 2, 5, 3, 3, 3, 5, 5, 7, 7, 8, 1$.

Escribe un programa en fortran chamado `exame3.f90` que defina o subprograma `analizaArquivo()`, debes decidir os argumentos axeitados, que lea o arquivo `datos3.txt` e calcule: 1) a suma dos números sempre que coincidan (sexan o mesmo) en dúas liñas consecutivas (debe sumar números mentres que a suma sexa menor que 10 ou se remate o arquivo); 2) o número de elementos sumados.

O programa principal ten que chamar ó subprograma `analizaArquivo()` e visualizar na pantalla o valor da suma (s) e o número de elementos sumados(n). Logo, ten que crear unha matriz **a** de orde n definida por:

$$a_{ij} = \frac{(ji)^2}{\sqrt{(j-i)^2+1}} \qquad i, j = 1, \ldots, n$$

Visualiza na pantalla a matriz **a** mostrando unha fila da matriz en cada liña.

NOTA: utilizando os datos do exemplo obteríase un $s = 12$ e un $n = 4$, ya que se sumarían os números $1, 3, 3, 5$.

SOLUCIÓN:

```fortran
program exame3
integer::s
real, dimension(:,:), allocatable :: a
real, parameter :: pi=3.14159
call analizaArquivo(s, n)
print*, "Suma= ", s, " Numero: ", n
allocate(a(n,n))
do i=1,n
    do j=1,n
            a(i,j)= float((i*j)**2)/sqrt((float(j-i)**2+1))
    end do
end do
print*, "Matriz a:"
do i=1,n
    print *, (a(i,j), j=1,n)
end do
deallocate(a)
stop
end program exame3

!*************************************************************
subroutine analizaArquivo(suma, numero)
integer, intent(out):: suma, numero
open(1, file="datos3.txt", status="old", err=3)
suma=0; numero=0
read(1,*, end=4) n
do
    if (suma >= 10) then
        close(1)
        exit
    end if
    read(1,*, end=4) m
    if( m == n) then
        suma = suma + n
        numero= numero + 1
    end if
    n=m
end do
4 close(1)
```

```
return
3 print*, "Erro abrindo datos3.txt"
end subroutine analizaArquivo
```

Control de Fortran

Escribe un programa en fortran chamado `exame4.f90` que pida ó usuario un número real a e cree un vector de números enteiros **x** con $n = 30$ elementos, x_i dado por:

$$x_i = \left\lfloor 10\cos^2\left(\frac{a\pi i}{4}\right)\right\rfloor \qquad i = 1, \ldots, n$$

Define un subprograma chamado `contarOcurrencias()`, cos argumentos axeitados, que dado un número enteiro m e un vector **z**, calcule o número de veces que se repite m no vector z. No programa principal, calcula o valor máximo de **x** (mx) e crea un vector **v** de dimensión $mx+1$ onde cada elemento v_i, $i = 0, \ldots, mx$ calcúlase chamando repetidamente ó subrograma `contarOcurrencias()` e será o número de veces que se repite o número i no vector **x**. Calcula tamén o número de elementos do vector **x** que hai que sumar para que a suma supere o valor 1000, tendo en conta que cando se chegue ó final do vector vólvese a comezar no principio. Garda este valor no arquivo `resultados4.txt` e os vectores **x** e **v**.

NOTA: se utilizas a =2.5 obtés o seguintes resultados:

```
Numero elementos sumados: 220
 1 5 8 0 8 4 1 10 1 5 8 0 8 4 1 10 1 5 8 0 8 4 1 10 1 5 8 0 8 4
 4 7 0 0 4 4 0 0 8 0 3
```

SOLUCIÓN:

```fortran
program exame4
integer, dimension(30):: x
integer, dimension(:), allocatable :: v
integer :: contarOcurrencias
real, parameter :: pi=3.14159
print '("a= ",$)'
read*, a
n=30
mx=0
do i=1, n
    x(i)=int(10*cos(a*pi*i/4)**2)
    if(x(i) > mx) mx=x(i)
end do
allocate(v(0:mx))
do i=0, mx
    v(i)=contarOcurrencias(x, n, i)
end do
suma=0
i=1
nc= 0
do
    suma = suma + x(i)
    nc= nc + 1
    if (suma > 1000) exit
    i= i+ 1
    if( i > n) i=1
end do
open(2, file="resultados4.txt", status="new", err=8)
write(2, '("Numero elementos sumados: ", i0)') nc
write(2, *) (x(i), i=1,n)
write(2, *) (v(i), i=0,mx)
close(2)
deallocate(v)
stop
8 print*, "Erro abrindo arquivo resultados4.txt"
end program exame4

!*****************************************************
```

```fortran
integer function contarOcurrencias(z,n, m)
integer, dimension(n), intent(in) :: z
integer, intent(in) :: n, m
k=0
do i=1,n
    if (z(i) == m) k= k+1
end do
contarOcurrencias=k
return
end function contarOcurrencias
```

Control de Fortran

Utilizando o editor de texto crea o arquivo `datos5.txt` que contén a dimensión e os valores de dous vectores: **x** de dimensión m e **y** de dimensión n (primeira liña m, segunda liña **x**, terceira liña n e cuarta liña **y**) con $m < n$. Por exemplo, `datos5.txt` podería conter os datos:

```
3
3 6 4
10
1 3 4 2 5 8 3 6 4 7
```

Escribe un programa en fortran chamado `exame5.f90` que visualice na pantalla os vectores **x** e **y** e amose unha mensaxe informando se o vector **x** está contido ou non no vector **y** utilizando o subprograma `estaContido()`. Este subprograma tes que definilo ti cos argumentos axeitados.

Calcula a suma dos elementos dos vectores **x** e **y** que serán respectivamente sx e sy. Calcula tamén o número de veces que hai que dividir por 2 o valor sy para que sy sexa menor que sx.

NOTA: co exemplo anterior terías que obter que **x** está contido en **y**, e o número de veces sería 2.

SOLUCIÓN:

```fortran
program exame5
integer, dimension(:), allocatable :: x, y
logical :: estaContido
open(8, file="datos5.txt", status="old", err=1)
read(8,*) m
allocate(x(m))
read(8,*) (x(i), i=1,m)
read(8,*) n
allocate(y(n))
read(8,*) (y(i), i=1,n)
close(8)
print*, "x= ", (x(i), i=1,m)
print*, "y= ", (y(i), i=1,n)

if (estaContido(x,m,y,n)) then
    print*, "Vector x contido no vector y"
else
    print*, "Vector x non contido en vector y"
end if
sx=sum(x)
sy=sum(y)
nv=0
do
    if (sy < sx) exit
    sy = sy/2
    nv=nv +1
end do
print*, "Numero de veces: ", nv
deallocate(x,y)
stop
1 print*, "Erro abrindo arquivo datos5.txt"
end program exame5

!****************************************
logical function estaContido(x,m, y, n)
integer, dimension(m), intent(in) :: x
integer, dimension(n), intent(in) :: y
integer, intent(in) :: n, m
estaContido=.false.
do i=1, n-m
```

```
    estaContido=.true.
    do j=1, m
        if (x(j) /= y(i+j)) then
            estaContido= .false.
            exit
        end if
    end do
    if (estaContido) return
end do
return
end function estaContido
```

Control de Fortran

Escribe un arquivo de texto chamado `matriz.dat` co seguinte contido:

```
1 2 3 4 5
6 7 8 9 8
7 6 5 4 3
```

Logo, escribe un programa en fortran chamado `exame.f90` que chame a un subprograma `le_matriz(...)`, do tipo axeitado e pasándolle os argumentos necesarios para que lea a matriz do arquivo anterior. Usa unha matriz estática enteira **a** de orde $n \times m$, con n, m constantes iguais a 3 e 5, respectivamente. O programa principal debe calcular a suma s dos elementos de **a**. Logo, debe ir sumando os seus elementos ata que a suma dos elementos sumados supere $s/2$. Os elementos sumados deben almacenarse nun vector **x**, que se debe mostrar na terminal. Finalmente, o programa debe calcular a partir de **a** unha matriz **b**, tamén de orde $n \times m$, con elementos b_{ij}, para $i = 1, \ldots, n$ e $j = 1, \ldots, m$, definidos como:

$$b_{ij} = \sum_{k=1}^{l} a_{ik} a_{kj} \tag{1}$$

sendo $l = \text{mín}(n, m)$. O programa debe mostrar na terminal a matriz **b** con formato de ancho 5.

SOLUCIÓN:

```fortran
program exame15
integer,parameter :: n=3,m=5
integer,dimension(n,m) :: a,b
integer,dimension(n*m) :: x
call le_matriz(a,n,m)
s=sum(a)/2;s1=0;k=0;l=min(n,m)
filas: do i=1,n
  do j=1,m
    s1=s1+a(i,j);k=k+1;x(k)=a(i,j)
    if(s1>s) exit filas
  end do
end do filas
print *,'elementos sumados:',x(1:k)
print *,'b='
do i=1,n
  do j=1,m
    s=0
    do k=1,l
      s=s+a(i,k)*a(k,j)
    end do
    b(i,j)=s
    print '(i5,$)',b(i,j)
  end do
  print *,''
end do
stop
end program exame15

!_____________________________
subroutine le_matriz(a,n,m)
integer,dimension(n,m),intent(out) :: a
integer,intent(in) :: n,m
open(1,file='matriz.dat',status='old',err=1)
do i=1,n
  read (1,*) (a(i,j),j=1,m)
end do
close(1)
```

```
return
1 stop 'erro en open abrindo matriz.dat'
end subroutine le_matriz
```

Control de Fortran

Escribe un programa en Fortran chamado `exame.f90` que lea por teclado un número enteiro n e reserve memoria para dous vectores enteiros $\mathbf{x}$ e $\mathbf{y}$, ambos de lonxitude n. O programa debe ler $\mathbf{x}$ por teclado e chamar ao subprograma `calcula_y(...)`, do tipo e cos argumentos axeitados, que calcule os elementos y_i, con $i = 1, \ldots, n$, do vector $\mathbf{y}$, dados por $y_1 = 1$ e:

$$y_i = \begin{cases} 1 & x_i > x_{i-1} \\ -1 & \text{noutro caso} \end{cases} \quad , i = 2, \ldots, n$$

Cando retorne o subprograma, o programa principal debe mostrar por pantalla o vector $\mathbf{y}$, nunha soa liña. Logo, o programa principal debe percorrer o vector $\mathbf{y}$, sen sairse polo seu extremo superior, almacenando no arquivo `vectores.dat` os elementos x_i e y_i con ancho de campo 5, ata que o número de elementos $y_i = -1$ sexa maior que $\lfloor n/2 \rfloor$. Proba con $n = 5$ e $\mathbf{x} = (5, 2, 1, 7, 6)$.

SOLUCIÓN:

```fortran
program exame2
integer,dimension(:),allocatable :: x,y
print '("n? ",$)';read *,n
allocate(x(n),y(n))
print '("x? ",$)';read *,x
call calcula_y(x,y,n)
print *,'y=',y
open(1,file='vectores.dat',status='new',err=1)
j=0;k=n/2
do i=1,n
  if(y(i)==-1) j=j+1
  write (1,'(i5,1x,i5)') x(i),y(i)
  if(j>=k) exit
end do
close(1)
deallocate(x,y)
stop
1 stop 'erro: vectores.dat xa existe'
end program exame2

!-----------------------------------------------
subroutine calcula_y(x,y,n)
integer,dimension(n),intent(in) :: x
integer,dimension(n),intent(out) :: y
integer,intent(in) :: n
y(1)=1
do i=2,n
  if(x(i)>x(i-1)) then
    y(i)=1
  else
    y(i)=-1
  end if
end do
return
end subroutine calcula_y
```

Control de Fortran

Crea un arquivo de texto chamado `vector_matriz.dat` co seguinte contido:

```
4
2 7 1 9
3 1 5 6
2 4 10 7
1 9 5 6
7 3 1 8
```

Escribe un programa en Fortran chamado `exame.f90` que abra este arquivo e lea o número da primeira liña á variábel n. Logo, debe reservar memoria para un vector **x** de lonxitude n, e para dúas matrices **a** e **b** cadradas de orde n, ambas enteiras. A segunda liña do arquivo debe lerse ao vector **x**, e as 4 liñas restantes deben lerse á matriz **a**. O programa debe invocar a un subprograma chamado `calcula_b(...)`, do tipo e cos argumentos axeitados, que calcule a matriz **b** con elementos b_{ij}, con $i, j = 1, \ldots, n$, de modo que b_{ij} sexa o número de elementos da columna j da matriz **a** que hai que sumar para superar x_i. Se non se chega a cumprir a condición debe ser $b_{ij} = n$. Finalmente, o programa principal debe mostrar por pantalla a matriz **b**, con formato enteiro de ancho 8.

SOLUCIÓN:

```fortran
program exame36
integer,dimension(:),allocatable :: x
integer,dimension(:,:),allocatable :: a,b
open(1,file='vector_matriz.dat',status='old',err=1)
read (1,*) n
allocate(x(n),a(n,n),b(n,n))
read (1,*) x
do i=1,n
  read (1,*) (a(i,j),j=1,n)
end do
close(1)
call calcula_b(x,a,b,n)
print *,'matriz b:'
do i=1,n
  do j=1,n
    print '(i8," ",$)',b(i,j)
  end do
  print *,''
end do
deallocate(x,a,b)
stop
1 stop 'erro: vector_matriz.dat non existe'
end program exame36

!_____________________________________
subroutine calcula_b(x,a,b,n)
integer,dimension(n),intent(in) :: x
integer,dimension(n,n),intent(in) :: a
integer,dimension(n,n),intent(out) :: b
integer,intent(in) :: n
do i=1,n
  do j=1,n
    s=0
    do k=1,n
      s=s+a(k,j)
      if(s>=x(i)) exit
    end do
```

```
    b(i,j)=k
  end do
end do
return
end subroutine calcula_b
```

Control de Fortran

Crea co editor Kate un arquivo de texto chamado `vector_numeros.dat` co seguinte contido (cada número nunha liña distinta):

```
8 2 3 4 5 6 1 7 5 3 6 3 2 1
```

Escribe un programa en Fortran chamado `exame.f90` que declare un vector enteiro **x** de $n = 10$ (declara n como unha constante) elementos e lea números dende o arquivo `vector_numeros.dat` ata que se acade o final do vector ou o final do arquivo (usando o arquivo anterior, acadarase antes o final do vector). Logo, debe chamar ao subprograma `busca(...)`, do tipo e cos argumentos axeitados, que calcule o valor mínimo de k tal que:

$$\sum_{j=1}^{k} x_j > \sum_{j=k+1}^{n} x_j \tag{1}$$

Logo de chamar ao subprograma `busca(...)`, o programa principal debe mostrar por pantalla o valor de k calculado. Finalmente, o programa principal debe calcular unha matriz **a**, cadrada de orde n, con elementos reais a_{ij} dados por:

$$a_{ij} = \begin{cases} \dfrac{1}{i}\displaystyle\sum_{l=1}^{i} x_l & i > j \\ \dfrac{1}{j}\displaystyle\sum_{l=n-j+1}^{n} x_l & \text{noutro caso} \end{cases}$$

O programa debe mostrar a matriz **a**, cada fila nunha liña da terminal, con formato real de anchura 8 e 3 decimais.

SOLUCIÓN:

```fortran
program exame4
integer,parameter :: n=10
integer,dimension(n) :: x
real,dimension(n,n) :: a
integer :: busca
open(1,file='vector_numeros.dat',status='old',err=1)
do i=1,n
  read (1,*,end=2) x(i)
end do
2 close(1)
print *,'x=',x
k=busca(x,min(i,n))
print *,'num. elementos sumados= ',k
print *,'a='
do i=1,n
  do j=1,n
    if(i>j) then
      a(i,j)=sum(x(1:i))/real(i)
    else
      a(i,j)=sum(x(n-j+1:n))/real(j)
    end if
    print '(f8.3,$)',a(i,j)
  end do
  print *,''
end do
stop
1 stop 'erro: vector_numeros.dat non existe'
end program exame4
```

!————————————————————————————————————

```fortran
integer function busca(x,n) result(i)
integer,dimension(n),intent(in) :: x
integer,intent(in) :: n
s1=0;s2=sum(x)
do i=1,n
  t=x(i);s1=s1+t;s2=s2-t
  print *,i,x(i),s1,s2
  if(s1>s2) exit
end do
return
end function busca
```

Control de Fortran

Escribe co editor de textos un arquivo chamado `datos_exame.dat` co seguinte contido:

```
2 4
1 2 3 4
5 6 7 8
```

Logo escribe un programa en Fortran chamado `exame.f90` que lea os valores da primeira liña do arquivo `datos_exame.dat`, os almacene nas variábeis n e m e reserve memoria para unha matriz **a** de orde $n \times m$ con valores enteiros que se deben ler dende a liña 2 en adiante no arquivo anterior. O programa principal debe chamar ao subprograma `calcula(...)`, do tipo e cos argumentos oportunos. Este subprograma debe percorrer por filas os elementos da matriz **a** sumando os seus valores ata que esta suma k supere o valor 10, verificando que non se sae da matriz **a**. O programa principal debe crear un arquivo chamado `resultados_exame.dat` e almacenar k na súa primeira liña e os elementos impares de **a** na segunda liña, ambos co ancho de campo mínimo necesario.

```fortran
program exame16
interface
  integer function calcula(a) result(k)
    integer,intent(in) :: a(:,:)
  end function
end interface
integer,allocatable :: a(:,:)
open(1,file='datos_exame1.dat',status='old',err=1)
read (1,*) n,m
allocate(a(n,m))
do i=1,n
  read (1,*) (a(i,j),j=1,m)
end do
close(1)
k=calcula(a)
open(1,file='resultado_exame1.dat',status='new',err=2)
write (1,'(i0)') k
do i=1,n
  do j=1,m
    if(mod(a(i,j),2)==1) write (1,'(i0," ",$)') a(i,j)
  end do
end do
close(1)
deallocate(a)
stop
1 stop 'datos_exame1.dat non existe'
2 stop 'resultado_exame1.dat xa existe'
end program exame16
!————————————————————————————
integer function calcula(a) result(k)
integer,intent(in) :: a(:,:)
n=size(a,1);m=size(a,2);k=0
```

```
filas: do i=1,n
  do j=1,m
    k=k+a(i,j)**2
    if(k>10) exit filas
  end do
end do filas
return
end function calcula
```

Control de Fortran

Escribe un programa en Fortran chamado `exame.f90` que lea por teclado un número enteiro n (usa $n = 5$) e logo cree un vector $\mathbf{x}$ con n valores enteiros $x_i = i$ para $i = 1, \ldots, n$. O programa debe chamar ao subprograma `calcula(...)`, co tipo e argumentos axeitados, que calcule a matriz de Vandermonde $\mathbf{a}$ e o seu determinante m para o vector $\mathbf{x}$, dados polas expresións:

$$\mathbf{a} = \begin{bmatrix} 1 & x_1 & x_1^2 & \ldots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \ldots & x_2^{n-1} \\ \ldots & \ldots & \ldots & \ldots & \ldots \\ 1 & x_n & x_n^2 & \ldots & x_n^{n-1} \end{bmatrix}, \quad m = \prod_{i=1}^{n} \prod_{j=i+1}^{n} (x_j - x_i)$$

É decir, $a_{ij} = x_i^{j-1}$ para $i, j = 1, \ldots, n$. O programa principal debe crear o arquivo `vandermonde.dat` e almacenar nel $\mathbf{x}$, o determinante m e a matriz $\mathbf{a}$, cada fila nunha liña. Tódolos valores deben ter o ancho mínimo necesario.

```
program exame2
integer,allocatable :: x(:),a(:,:)
interface
  subroutine calcula(x,a,m)
    integer,intent(in) :: x(:)
    integer,intent(out) :: a(:,:),m
  end subroutine calcula
end interface
print '("n? ",$)';read *,n
allocate(x(n),a(n,n))
forall(i=1:n) x(i)=i
call calcula(x,a,m)
open(1,file='vandermonde.dat',status='new',err=1)
do i=1,n
  write (1,'(i0," ",$)') x(i)
end do
write (1,*)
write (1,'("determinante=",i0)') m
do i=1,n
  do j=1,n
    write (1,'(i0," ",$)') a(i,j)
  end do
  write (1,*)
end do
close(1)
deallocate(x,a)
stop
1 stop 'arquivo vandermonde.dat xa existe'
end program exame2
!-----------------------------------------------
subroutine calcula(x,a,m)
integer,intent(in) :: x(:)
```

```fortran
integer,intent(out) :: a(:,:),m
n=size(a,1);m=1
do i=1,n
  xi=x(i)
  forall(j=1:n) a(i,j)=xi**(j-1)
  do j=i+1,n
    m=m*(x(j)-xi)
  end do
end do
return
end subroutine calcula
```

Control de Fortran

Escribe co editor o arquivo `datos_exame3.dat` co seguinte contido: `6 3 2 1 4 7 8 5 0 1 5`, cada número nunha liña. Escribe un programa en Fortran chamado `exame.f90` que declare dous vectores enteiros **v** e **w**, ambos de lonxitude 10, e logo lea por teclado dous números enteiros n e m ambos menores que 10, pedíndoos de novo se algún deles é igual ou maior ca 10 (usa $n = 5, m = 3$). O programa debe chamar ao subprograma `lectura(...)`, do tipo e cos argumentos axeitados, que lea números dende o arquivo `datos_exame3.dat` ata $n + m$ valores ou ata que se remate o arquivo, almacenando os valores pares en **v** e os impares en **w**. Se j e k son os números de elementos pares e impares, respectivamente, o programa principal debe mostrar por pantalla os j primeiros elementos de **v** nunha liña, os k primeiros elementos de **w** na seguinte liña, e a suma de todos eles nunha terceira liña.

```fortran
program exame3
integer v(10),w(10)
interface
  subroutine lectura(v,w,p,j,k)
    integer,intent(out) :: v(:),w(:)
    integer,intent(in) :: p
    integer,intent(out) :: j,k
  end subroutine
end interface
do
  print '("n,m(<10)? ",$)';read *,n,m
  if(n<10.and.m<10) exit
end do
call lectura(v,w,n+m,j,k)
print *,'v=',v(1:j)
print *,'w=',w(1:k)
print *,'s=',sum(v(1:j))+sum(w(1:k))
stop
end program exame3
!-----------------------------------
subroutine lectura(v,w,p,j,k)
integer,intent(out) :: v(:),w(:)
integer,intent(in) :: p
integer,intent(out) :: j,k
j=0;k=0
open(1,file='datos_exame3.dat',status='old',err=1)
do i=1,p
  read (1,*,end=2) m
  if(mod(m,2)==0) then
    j=j+1;v(j)=m
  else
    k=k+1;w(k)=m
  end if
end do
2 close(1)
```

```
return
1 stop 'arquivo datos_exame3.dat non existe'
end subroutine lectura
```

Control de Fortran

Escribe co editor de textos un arquivo chamado `datos_exame4.dat` co seguinte contido (cada número nunha liña): `2.3 4.6 5.1 7.2 8.3 2.5`. Escribe un programa en Fortran chamado `exame.f90` que declare un vector **x** de 10 elementos e lea números dende este arquivo, almacenando os seus valores nos elementos de **x** ata que a suma dos números lidos supere 20, se exceda o número de elementos de **x** ou se remate o arquivo. En cada iteración, o programa debe mostrar o número de iteración, o elemento lido e o valor da suma nesa iteración. Chamaremos n ao número de valores finalmente lidos. O programa debe reservar memoria para unha matriz **a** cadrada de orde n. Logo, debe chamar ao subprograma `calcula(...)`, do tipo e cos argumentos axeitados, que calcule os elementos de **a** como $a_{ij} = ijx_k$, con $i, j = 1, \ldots, n$, onde k é 1 máis o resto de dividir $i^2 + j$ entre n. Finalmente, o programa debe crear un novo arquivo `resultado_exame4.dat` e almacenar nel os valores $x_i \sum_{j=1}^{n} a_{ij}$ para $i = 1, \ldots, n$, un valor en cada liña.

```
program exame4
interface
  subroutine calcula(x,n,a)
    real,intent(in) :: x(:)
    integer,intent(in) :: n
    real,intent(out) :: a(n,n)
  end subroutine calcula
end interface
real x(10)
real,allocatable :: a(:,:)
open(1,file='datos_exame4.dat',status='old',err=1)
n=0;s=0
do
  n=n+1;read (1,*,end=3) x(n)
  s=s+x(n)
  print *,n,x(n),s
  if(s>20.or.n>10) exit
end do
3 close(1)
allocate(a(n,n))
call calcula(x,n,a)
open(1,file='resultado_exame4.dat',status='new',err=2)
do i=1,n
  write (1,*) x(i)*sum(a(i,:))
end do
close(1)
deallocate(a)
stop
1 stop 'arquivo datos_exame4.dat non existe'
2 stop 'arquivo resultado_exame4.dat xa existe'
end program exame4
!————————————————————————————
subroutine calcula(x,n,a)
real,intent(in) :: x(:)
integer,intent(in) :: n
real,intent(out) :: a(n,n)
do i=1,n
  do j=1,n
    k=1+mod(i*i+j,n);a(i,j)=i*j*x(k)
  end do
end do
return
end subroutine calcula
```

Control de Fortran

Escribe co editor un arquivo chamado `datos_exame5.dat` co seguinte contido: `3; 1 2 3; 4 5 6; 7 8 9; 9 8 7; 6 5 4; 3 2 1`, onde o símbolo `;` separa liñas distintas. Escribe un programa en Fortran chamado `exame.f90` que lea o número n da primeira liña e reserve memoria para 3 matrices enteiras **a**, **b** e **c** cadradas de orde n. Logo, debe ler as liñas restantes do arquivo e almacenalas nas matrices **a** e **b** (n liñas cada unha). Seguidamente, debe chamar ao subprograma `calcula(...)`, do tipo e cos argumentos axeitados, que calcule os elementos da matriz **c** do seguinte modo: para calcular c_{ij}, con $i, j = 1, \ldots, n$, suma os elementos da fila i de **a** e da columna j de **b** (un elemento de cada unha en cada iteración) rematando cando a suma supere o producto $a_{ij}b_{ij}$, de modo que c_{ij} sexa o número de iteracións. Finalmente, o programa principal debe mostrar por pantalla, para cada fila de **c**, o producto dos cadrados dos seus elementos (cada producto nunha liña distinta da terminal).

```fortran
program exame5
integer,allocatable :: a(:,:),b(:,:),c(:,:)
interface
  subroutine calcula(a,b,c)
    integer,intent(in) :: a(:,:),b(:,:)
    integer,intent(out) :: c(:,:)
  end subroutine
end interface
open(1,file='datos_exame5.dat',status='old',err=1)
read (1,*) n
allocate(a(n,n),b(n,n),c(n,n))
do i=1,n
  read (1,*) (a(i,j),j=1,n)
end do
do i=1,n
  read (1,*) (b(i,j),j=1,n)
end do
close(1)
call calcula(a,b,c)
print *,'c='
do i=1,n
  print *,(c(i,j),j=1,n)
end do
do i=1,n
  print *,product(c(i,:)**2)
end do
deallocate(a,b,c)
stop
1 stop 'arquivo datos_exame5.dat non existe'
end program exame5
!-----------------------------------------
subroutine calcula(a,b,c)
integer,intent(in) :: a(:,:),b(:,:)
integer,intent(out) :: c(:,:)
```

```
n=size(a,1)
do i=1,n
  do j=1,n
    l=0;m=a(i,j)*b(i,j)
    do k=1,n
      l=l+a(i,k)+b(k,j)
      if(l>m) exit
    end do
    c(i,j)=k
  end do
end do
return
end subroutine calcula
```

Control de Fortran

Crea un arquivo chamado `datos_exame.dat` co seguinte contido (cada número nunha fila):

```
8 2 1 4 9
```

Escribe un programa chamado `exame.f90` que declare un vector **x** e unha matriz **a** dinámicos con valores enteiros e abra o anterior arquivo `datos_exame.dat`, conte o número n de liñas e reserve memoria para **x** e **a**, ambos de orde n. Logo, o programa debe ler os valores no arquivo e almacenalos no vector **x**, mostrando por pantalla **x** nunha única liña. Despois, debe chamar ao subprograma `matriz(...)`, do tipo e cos argumentos axeitados, que calcule **a** de modo que a_{ij}, con $i, j = 1, \ldots, n$, sexa o número de valores enteiros, comezando en 1 e aumentando de 1 en 1, que hai que sumar para superar $i^2 j$. Finalmente, o programa principal debe mostrar a matriz **a** na terminal, unha fila en cada liña da terminal, cada elemento co ancho mínimo necesario.

```
program exame
integer,allocatable :: x(:),a(:,:)
n=0
open(1,file='datos_exame.dat',status='old',err=1)
do
    read (1,*,end=2);n=n+1
end do
2 allocate(x(n),a(n,n));rewind(1)
do i=1,n
    read (1,*) x(i)
end do
close(1)
print *,'x=',x
call matriz(a,n)
print *,'a='
do i=1,n
    do j=1,n
        print '(i0," ",$)',a(i,j)
    end do
    print *,''
end do
deallocate(x,a)
stop
1 stop 'arquivo datos_exame.dat non atopado'
end program exame
!————————————————————————————————————
subroutine matriz(a,n)
integer,intent(out) :: a(n,n)
integer,intent(in) :: n
integer ::p
do i=1,n
    do j=1,n
        p=i**2*j;k=1;l=0
        do
```

```
            l=l+k;k=k+1
            if(l>p) exit
        end do
        a(i,j)=l
    end do
end do
end subroutine matriz
```

Control de Fortran

Escribe un programa en Fortran chamado `exame.f90` que lea dos números enteiros n e m por teclado (usa n=2 e m=5) e calcule $p = n+m$ e $q = nm$. Logo, debe reservar memoria para unha matriz cadrada **a** de orde p e para un vector **v** de lonxitude q. O programa debe calcular v_i, con $i = 1, \ldots, q$, como o número de múltiplos de 5 que hai entre i e qi. Logo, debe mostrar **v** por pantalla na mesma liña e chamar ao subprograma `matriz(...)`, do tipo e cos argumentos axeitados, que calcule a_{ij}, con $i, j = 1, \ldots, p$, como:

$$a_{ij} = \sum_{k=1}^{j} i v_{k\,\%q+1} \tag{1}$$

onde $k\,\%q$ é o resto da división enteira de k entre q. O programa principal debe mostrar por pantalla **v** nunha única liña. Finalmente, debe almacenar no arquivo `saida_exame.dat` a matriz **a**, cada fila nunha liña do arquivo con formato enteiro de ancho 4.

```
program exame
integer :: p,q
integer,allocatable :: a(:,:),v(:)
print '("n,m? ",$)';read *,n,m  ! n=2,m=5
p=n+m; q=n*m
allocate(a(p,p),v(q))
do i=1,q
  s=0
  do l=i,q*i
    if(mod(l,5)==0) s=s+1
  end do
  v(i)=s
end do
print *,'v=',v
call matriz(a,p,v,q)
open(1,file='saida_exame.dat',status='new',err=1)
do i=1,p
  do j=1,p
    write (1,'(i4," ",$)') a(i,j)
  end do
  write (1,*) ''
end do
close(1)
deallocate(a)
stop
1 stop 'saida_exame.dat xa existe'
end program exame
!-----------------------------
subroutine matriz(a,p,v,q)
integer,intent(in) :: p,q,v(q)
integer,intent(out) :: a(p,p)
do i=1,p
  do j=1,p
```

```
      n=0
      do k=1,j
         l=mod(k,q)+1;n=n+i*v(l)
      end do
      a(i,j)=n
   end do
end do
end subroutine matriz
```

Control de Fortran

Escribe co editor de textos un arquivo chamado `datos_exame.dat` co seguinte contido:

```
4 3
1 2 3
4 5 6
7 8 9
8 7 6
```

Escribe un programa en Fortran chamado `exame.f90` que chame a un subprograma `le_matriz(...)`, do tipo e cos argumentos axeitados, que lea dende o arquivo anterior os números n e m da primeira fila, reserve memoria para unha matriz **a** enteira de n filas e m columnas, e lea os restantes números do arquivo aos elementos a_{ij} con $i, \ldots, n$ e $j = 1, \ldots, m$. Logo, o programa principal debe percorrer a matriz **a** por filas, sumando os elementos a_{ij} pares na variábel k e os múltiplos de 3 na variábel l ata que $kl \geq p$, sendo:

$$p = \left(\sum_{i=1}^{n} \sum_{j=1}^{m} a_{ij} \right)^2 \tag{1}$$

Cando se acade o final da matriz, o programa ten que continuar polo comezo da mesma. O programa debe mostrar, en cada iteración, os valores i, j, a_{ij}, k, l, kl e p. Finalmente, o programa debe liberar a memoria da matriz **a**.

```fortran
program exame
interface
  function le_matriz(nf) result(a)
    character(*),intent(in) :: nf
    integer,allocatable :: a(:,:)
  end function le_matriz
end interface
integer :: p,q
integer,allocatable :: a(:,:)
a=le_matriz('datos_exame.dat')
n=size(a,1);m=size(a,2);k=0;l=0;p=sum(a)**2
iter: do
  do i=1,n
    do j=1,m
      q=a(i,j)
      if(mod(p,2)==0) k=k+q
      if(mod(p,3)==0) l=l+q
      print *,i,j,q,k,l,k*l,p
      if(k*l>=p) exit iter
    end do
  end do
end do iter
deallocate(a)
end program exame

function le_matriz(nf) result(a)
```

```fortran
character(*),intent(in) :: nf
integer,allocatable :: a(:,:)
open(1,file=nf,status='old',err=1)
read (1,*) n,m
allocate(a(n,m))
do i=1,n
  read (1,*) (a(i,j),j=1,m)
end do
close(1)
return
1 print *,nf,'non existe';stop
end function le_matriz
```

Control de Fortran

Escribe un programa en Fortran chamado `exame.f90` que lea por teclado un número n verificando que sexa $5 \leq n \leq 10$, e volvendo a pedir n en caso de que non se cumpran estas condicións. O programa debe reservar memoria para dous vectores $\mathbf{x}$ e $\mathbf{y}$ con n elementos reais cada un, definidos por:

$$x_i = \frac{n^2 i + i^2 + 1}{n^2 + i^2}, \quad y_i = \sum_{j=1}^{i} x_j \tag{1}$$

e mostrar por pantalla, na mesma liña, os elementos x_i e y_i co mesmo i, para $i = 1, \ldots, n$. Logo, o programa debe chamar a un subprograma `matriz(...)`, do tipo e cos argumento axeitados, que calcule os elementos dunha matriz enteira cadrada $\mathbf{a}$ de orde n. O elemento a_{ij} debe ser a suma de: 1) o número de elementos do vector $\mathbf{x}$ inferiores a $2i + j$; e 2) o número de elementos do vector $\mathbf{y}$ superiores a $2i + j$. Finalmente, o programa principal debe crear o arquivo `saida_exame.dat` e almacenar nel a matriz $\mathbf{a}$, con formato enteiro de ancho 3, cada fila nunha liña do arquivo.

```
program exame
real,allocatable :: x(:),y(:)
integer,allocatable :: a(:,:)
do
  print '("5<=n<=10? ")';read *,n
  if(n>=5.and.n<=10) exit
end do
allocate(x(n),y(n),a(n,n));s=0;n2=n**2
do i=1,n
  x(i)=(n2*i+i**2+1)/real(n2+i**2)
  s=s+x(i);y(i)=s
  print *,x(i),y(i)
end do
call matriz(a,x,y,n)
open(1,file='saida_exame.dat',status='new',err=1)
do i=1,n
  do j=1,n
    write (1,'(i0," ",$)') a(i,j)
  end do
  write (1,*) ''
end do
close(1)
deallocate(x,y,a)
stop
1 stop 'arquivo saida_exame.dat xa existe'
end program exame
!-----------------------------------
subroutine matriz(a,x,y,n)
integer,intent(out) :: a(n,n)
real,intent(in) :: x(n),y(n)
integer,intent(in) :: n
do i=1,n
```

```
  do j=1,n
    k=2*i+j ; a(i,j)=count(x<k)+count(y>k)
  end do
end do
end subroutine matriz
```

Control de Fortran

Crea co editor Kate o seguinte arquivo de texto `datos1.txt` coa seguinte matriz cadrada de valores enteiros:

```
2 5 2 7 9
2 1 4 3 5
3 2 4 5 6
1 2 3 5 1
6 1 4 7 2
```

Escribe un programa en Fortran que conte o número n de liñas do arquivo, reserve memoria para a matriz cadrada **a** de orde n e lea o arquivo `datos1.txt` a esta matriz. Logo, chama ao subprograma `producto(...)`, do tipo e cos argumentos axeitados, que calcule o producto da matriz **a** pola súa trasposta usando sentenzas de iteración. O resultado debe ser a matriz **d**, definida por:

$$d_{ij} = \sum_{k=1}^{n} a_{ik} a_{jk} \quad i, j = 1, \ldots, n \tag{1}$$

O programa principal debe visualizar na pantalla a matriz **d**, cada fila nunha liña da terminal. Logo, debe calcular a suma m dos elementos da matriz **a** (percorrida por filas) que son superiores aos seus correspondentes da matriz trasposta mentres que a suma sexa inferior a $\frac{1}{4}\sum_{i=1}^{n}\sum_{j=1}^{n} a_{ij}$ ou se acade o final da matriz **a**. Finalmente, o programa principal debe mostrar a suma m co formato enteiro de ancho mínimo.

```fortran
program exame1
integer,allocatable:: a(:,:), d(:,:)
open(1, file="datos1.txt", status="old",err=5)
n=0
do
    read(1,*, end=2);n=n+1
end do
2 allocate(a(n,n),d(n,n));rewind(1)
do i=1,n
  read(1,*) (a(i,j),j=1,n)
end do
close(1)
call producto(a,n,d)
print *, "d= "
do i=1,n
    print *, (d(i,j), j=1,n)
end do
k=sum(a)/4;m=0
filas: do i=1,n
  do j=1,n
    if(a(i,j)>a(j,i)) then
      m=m+a(i,j)
    end if
```

```
    if(m>k) exit filas
  end do
end do filas
print '("s=",i0)',m
deallocate(a,d)
stop
5 print*, "Erro lendo o arquivo datos1.txt"
end program exame1
!*****************************************************
subroutine producto(a,n,d)
integer,intent(in) :: a(n,n),n
integer,intent(out) :: d(n,n)
do i=1,n
    do j=1,n
!           t=0
!           do k=1,n
!                t=t+ a(i,k)*a(j,k)
!           end do
!           d(i,j)=t
        d(i,j)=dot_product(a(i,:),a(j,:))
    end do
end do
end subroutine producto
```

Control de Fortran

Escribe un programa en Fortran que lea un número enteiro n por teclado, reserve memoria para un vector $\mathbf{x}$ de dimensión n e lea por teclado os elementos do vector $\mathbf{x}$. Usa $n = 5$, $\mathbf{x} = (1, 2, 3, 4, 5)$. Logo, calcula outro vector $\mathbf{y}$ tamén de dimensión n, onde $y_1 = 0$ e:

$$y_i = \frac{x_i + y_{i-1}}{x_{n-i+1}^2} \quad i = 2, \ldots, n \tag{1}$$

Define un subprograma chamado `calculaVector(...)`, cos argumentos axeitados, que calcule o vector $\mathbf{w}$ de lonxitude n onde w_i, con $i = 1, \ldots, n$, para i par será o menor número p que verifique:

$$\sum_{j=1}^{p} \frac{x_j + y_j}{2} > \sum_{k=1}^{i} y_k \tag{2}$$

Se i é impar, w_i será o producto escalar dos vectores $\mathbf{x}$ e $\mathbf{y}$ ate a posición i, definido por:

$$w_i = \sum_{j=1}^{i} x_j y_j \tag{3}$$

Garda no arquivo `saida2.txt` os vectores $\mathbf{y}$ e $\mathbf{z}$, un vector en cada liña con formato de ancho 7 e dúas cifras decimais.

```
program exame2
real, allocatable :: x(:), y(:), w(:)
print *, "n= "
read *, n
allocate(x(n), y(n), w(n))
print *, "x= "
read *, x
y(1)=0
do i=2,n
    y(i) =(x(i)+y(i-1))/x(n-i+1)**2
end do
print *, "y= ", y
call calculaVector(x, y, w, n)
print*, "w= ", w
open(1,file='saida2.txt')
do i=1,n
  write (1,'(f7.2,$)') y(i)
end do
write (1,*)
do i=1,n
  write (1,'(f7.2,$)') w(i)
end do
close(1)
deallocate(x,y, w)
stop
```

```fortran
end program exame2

!*************************************
subroutine calculaVector(x, y, w, n)
real, intent(in) :: x(n), y(n)
real, intent(out) :: w(n)
integer, intent(in) :: n
integer :: p
do i=1,n
    if (mod(i,2) == 0) then
        p=0;s=0;si=sum(y(1:i))
        do j=1,n
            s = s + (x(j)+y(j))/2;p = p+1
            if (s > si) exit
        end do
        w(i) = p
    else
        w(i) = dot_product(x(1:i), y(1:i))
    end if
end do
return
end subroutine calculaVector
```

Control de Fortran

Escribe no editor duas liñas con $n = 8$ números enteiros e gárdao no arquivo `datos3.txt`. Por exemplo, co seguinte contido:

```
2 3 4 -5 7 2 5 9
2 4 1 1 10 6 4 -5
```

Escribe un programa en Fortran que lea o arquivo `datos3.txt`, comprobando os erros na lectura do arquivo, e garde no vector **x** a primeira liña e no vector **y** a segunda liña. Visualiza na pantalla os vectores. Define o subprograma `CalculaVector(...)`, do tipo e cos argumentos necesarios, que calcule un vector **z** cos elementos comúns de **x** e **y**, non incluíndo elementos repetidos. Desde o programa principal, visualiza na pantalla o vector **z**, o seu elemento máximo e a súa posición no vector **z**.

```fortran
program exame3
integer,parameter :: n=8
integer :: x(n),y(n),z(n),k
open(1,file='datos3.txt',status='old',err=1)
read (1,*) x
read (1,*) y
close(1)
print *,'x=',x
print *,'y=',y
call CalculaVector(x,y,z,n,k)
print *,'z=',z(1:k)
print '("max=",i0," imax=",i0)',maxval(z(1:k)),maxloc(z(1:k))
stop
1 stop 'erro lendo datos3.txt'
end program exame3
!-----------------------------------------
subroutine CalculaVector(x,y,z,n,k)
integer,intent(in) :: x(n),y(n),n
integer,intent(out) :: z(n),k
k=0
do i=1,n
  m=x(i)
  if(any(m==z(1:k))) cycle
  if(any(m==y)) then
    k=k+1;z(k)=m;
  end if
end do
end subroutine CalculaVector
```

Control de Fortran

Escribe no editor unha matriz de números enteiros e gárdaa no arquivo `datos4.txt`:

```
3 8
2 3 4 -5 7 2 5 9
2 4 1 1 10 6 4 -5
8 6 5 3 9 24 5 14
```

A primeira liña do arquivo son o número de filas e columnas da matriz e as seguintes liñas son os elementos da matriz. Escribe un programa en Fortran que lea o arquivo `datos4.txt` e o almacene na matriz **a**. Define un subprograma `calculaMatriz(...)`, decide os seus argumentos, que calcule unha matriz **b** das mesmas dimensións que **a**, onde cada elemento b_{ij}, está definido por:

$$b_{ij} = \begin{cases} \displaystyle\sum_{l=i-1}^{i} \sum_{k=j-1}^{j} a_{lk} & \text{se } i, j > 1 \\ a_{ij} & \text{resto dos casos} \end{cases}$$

Dende o programa principal, visualiza na pantalla a matriz **b**, cada fila nunha liña da terminal, e calcula a suma *sp* dos elementos pares de **b** percorrida por columnas mentres que a suma sexa menor que *sb*, sendo *sb* a suma dos elementos de **b** dividido por 10. Visualiza na pantalla *sp* e o número de elementos sumados.

```fortran
program exame4
integer, allocatable :: a(:,:), b(:,:)
open(1, file="datos4.txt", status="old", err=4)
read(1, *) nf, nc
allocate(a(nf, nc), b(nf, nc))
do i=1,nf
    read(1,*) (a(i,j), j=1,nc)
end do
close(1)
call calculaMatriz(a, b, nf, nc)
print*, "b= "
do i=1,nf
    print*, (b(i,j), j=1,nc)
end do
sp=0; np=0; sb=sum(b)/10
doexterno: do j=1,nc
    do i=1, nf
        k=b(i,j)
        if(mod(k,2)==1) cycle
        sp = sp + k; np = np +1
        if (sp > sb) exit doexterno
    end do
end do doexterno
print*, "sp= ", sp, " np= ", np
```

```
deallocate(a,b)
stop
4 print*, "Erro abrindo arquivo datos4.txt"
end program exame4

subroutine calculaMatriz(a, b, nf, nc)
integer, intent(in) :: a(nf,nc),nf,nc
integer, intent(out) :: b(nf, nc)
do i=1,nf
    do j=1,nc
        if (i > 1 .and. j>1) then
!               t=0
!               do l=i-1, i
!                   do k= j-1, j
!                       t=t+ a(l,k)
!                   end do
!               end do
!               b(i,j)=t
            b(i,j)=sum(a(i-1:i,j-1:j))
        else
            b(i,j)=a(i,j)
        end if
    end do
end do
return
end subroutine calculaMatriz
```

Control de Fortran

Escribe un programa en Fortran que lea por teclado un número enteiro n (usa $n = 5$). Lee por teclado n números enteiros e almacénaos no vector **x**. Introduce os elementos do vector **x** nunha matriz **a**, recheando por filas, de dimensión $m \times m$, onde $m = \lfloor \sqrt{n} \rfloor$. Os últimos $n - m^2$ elementos de **x** deben quedar fóra da matriz **a**. Visualiza a matriz **a** na pantalla (unha fila en cada liña) e calcula a súa traza. Define un subprograma `calculaVector(...)`, debes decidir o seu tipo e argumentos, que calcule os elementos de **x** que están repetidos e o número de veces que se repiten. Por exemplo se **x** contén os elementos `2 5 7 9 2 10 7 2 3 1`, a resposta sería que 2 está repetido 3 veces e 7 dúas veces. Desde o programa principal garda no arquivo `saida5.txt` en cada liña o elemento repetido e o número de veces que se repite.

```
program exame5
integer, allocatable :: x(:), xr(:), nx(:), a(:,:)
print*, "n= "
read*, n
allocate(x(n), xr(n), nx(n))
print*, "x= "
read*,x
m=floor(sqrt(real(n)));allocate(a(m,m))
k=1; tr=0
do i=1,m
    do j=1,m
        a(i,j) = x(k); k = k+1
    end do
    tr = tr + a(i,i)
end do
print*, "Traza de a: ", tr
call calculaVector(x, n, xr, nx, nf)
open(1, file="saida5.txt", status="new", err=4)
do i=1, nf
    write(1, *) xr(i), nx(i)
end do
close(1)
deallocate(x, xr, nx, a)
stop
4 print*, "Erro abrindo arquivo saida5.txt"
end program exame5

subroutine calculaVector(x, n, r, nv, nf)
integer, intent(in) :: x(n), n
integer, intent(out) :: r(n), nv(n), nf
nf=0
do i=1,n
    m = x(i);j=i-1;
    if(any(m==x(1:j))) cycle
    nveces = 1
    do j= i+1, n
```

```
        if (m == x(j)) nveces = nveces +1
    end do
    if (nveces>1) then
        nf = nf + 1;r(nf) = m;nv(nf) = nveces
    end if
end do
return
end subroutine calculaVector
```

Control de Fortran

Escribe un programa en Fortran que lea un número enteiro n por teclado (usa $n = 5$). Crea unha matriz **a** de dimensións $n \times n$ definida por:

$$a_{ij} = \begin{cases} \mathrm{e}^{-j}\,\mathrm{sen}\left(\dfrac{\pi i}{10}\right) & \text{se } i > j \\ ij\sqrt{i+j} & \text{se } i < j \\ 10ij & \text{se } i = j \end{cases}$$

e garda a matriz **a** no arquivo `saida6.txt`, cada fila nunha liña con formato real de ancho 7 e 2 decimais. Define un subprograma chamado `calculos(...)`, do tipo e cos argumentos axeitados, que sume os elementos da matriz **a**, percorrida por columnas, ata que a súa suma sexa superior a s, calculando como resultados o número de elementos sumados e o índice de fila e columna na que se parou de sumar. O valor de s é a suma dos elementos da diagonal secundaria da matriz **a** dados pola expresión:

$$s = \sum_{i=1}^{n} a_{i(n+1-i)} \tag{1}$$

Desde o programa principal, visualiza na pantalla os resultados do subprograma `calculos(...)`.

```
program exame5
real, allocatable :: a(:,:)
real, parameter :: pi=3.14159
print*, "n= "
read*, n
allocate(a(n,n))
do i=1,n
    do j=1,n
        if (i > j) then
            a(i,j) = sin(pi * i/10)*exp(real(-j))
        else if (i < j) then
            a(i,j) = i*j*sqrt(real(i+j))
        else
            a(i,j) = 10*i*j
        end if
    end do
end do
open(1, file="saida6.txt", status="new", err=4)
do i=1, n
  do j=1,n
    write (1,'(f7.2,$)') a(i,j)
  end do
  write (1,*)
end do
close(1)
call calculos(a, n, ne, nf, nc)
print*, "no. elementos= ", ne, " fila= ", nf, " columna= ", nc
```

```
deallocate(a)
stop
4 print*, "Erro abrindo arquivo saida6.txt"
end program exame5

subroutine calculos(a, n, ne, nf, nc)
real, intent(in) :: a(n,n)
integer, intent(in) :: n
integer, intent(out) :: nf, nc, ne
s=0
do i=1,n
    s = s + a(i, n+1-i)
end do
se=0; ne=0
filas: do j=1,n
    do i=1,n
        se = se + a(i,j); ne = ne + 1
        if (se > s) then
            nf = i; nc = j
            exit filas
        end if
    end do
end do filas
return
end subroutine calculos
```

Control de Fortran, CLI1, curso 2022-2023

Consideremos dous polinomios: $p(x) = \sum_{i=0}^{n} p_i x^i$, de grao n, e $q(x) = \sum_{j=0}^{m} q_j x^j$, de grao m. O producto $p \cdot q$ é un polinomio $r(x) = \sum_{k=0}^{l} r_k x^k$ de grao $l = n + m$, onde r_k está dado por:

$$r_k = \sum_{i=u}^{k} p_i q_{k-i}, \quad k = 0, \ldots, l \tag{1}$$

onde $u = \text{máx}(0, k - m)$. Crea co editor Kate o seguinte arquivo de texto `datos1.txt`:

```
2 3
1 2 1
1 0 -1 2
```

Escribe un programa en Fortran que abra o arquivo `datos1.txt` e lea os números da primeira liña, almacenándoos nas variábeis n e m. Reserva memoria para dous vectores **p** e **q**, con valores enteiros, de dimensións $n+1$ e $m+1$ e con elementos $\mathbf{p} = (p_0, \ldots, p_n)$ e $\mathbf{q} = (q_0, \ldots, q_m)$, respectivamente (coeficientes dos polinomios p e q). Ambos vectores deben ter o elemento con índice 0 como primeiro elemento. Le dende a segunda e terceira liña de `datos1.txt` os vectores **p** e **q**, respectivamente. Reserva tamén memoria para un terceiro vector **r** enteiro de lonxitude $l = n+m+1$. Chama repetidamente ao subprograma `coef(...)`, do tipo e cos argumentos axeitados, que calcule o coeficiente r_k do polinomio producto r seguindo a ecuación 1. Finalmente, o programa debe mostrar por pantalla os coeficientes de r. Debes obter **r**=(1,2,0,0,3,2).

```fortran
program cli1
integer,allocatable :: p(:),q(:),r(:)
integer :: coef
open(1,file='datos1.txt',status='old',err=1)
read (1,*) n,m
print *,'n=',n,' m=',m
allocate(p(0:n),q(0:m))
read (1,*) p
read (1,*) q
close(1)
print *,'p=',p
print *,'q=',q
l=n+m
allocate(r(0:l))
do k=0,l
    r(k)=coef(p,n,q,m,k)
end do
print *,'r=',r
deallocate(p,q,r)
stop
1 stop 'datos1.txt non existe'
end program cli1
!
```

```
integer function coef(p,n,q,m,k) result(r)
integer,intent(in) :: p(0:n),n,q(0:m),m,k
r=0
do i=max(0,k-m),k
    r=r+p(i)*q(k-i)
end do
end function coef
```

Control de Fortran, CLI2, curso 2022-2023

Escribe un programa en Fortran que defina unha constante enteira n=10 e tres vectores estáticos **x**=(1,7,3,9,0,1,2,3,2,1), **y** e **z**, todos de lonxitude n. O programa debe almacenar en **y** os valores de **x** sen repeticións. Tamén debe almacenar en **z** os índices dos elementos de **x** en **y**. Para isto, debes percorrer os elementos de **x**. En cada elemento x_i, comproba se x_i está en **y**. Se é así, almacena en z_i o índice do elemento y_i que é igual a x_i. Se x_i non está en **y**, engade x_i a **y**. O programa debe chamar a un subprograma `escribe(...)`, do tipo e cos argumentos axeitados, que almacene no arquivo `datos2.txt`, na súa liña i-ésima con $i = 1, \ldots, n$, o elemento x_i e o índice z_i co que aparece en **y**. Almacena **y** na última liña.

```fortran
program cli2
integer,parameter :: n=10
integer :: x(n)=(/1,7,3,9,0,1,2,3,2,1/),y(n),z(n)
l=0
do i=1,n
    u=x(i)
    do k=1,l
        if(u==y(k)) exit
    end do
    if(k<=l) then
        z(i)=k
    else
        l=l+1;y(l)=x(i);z(i)=l
    end if
end do
call escribe(x,y,z,n,l)
end program cli2
!-----------------------------------
subroutine escribe(x,y,z,n,l)
integer,intent(in) :: x(n),y(n),z(n),n,l
open(1,file='datos2.txt')
do i=1,n
    write (1,*) x(i),z(i)
end do
write (1,*) (y(i),i=1,l)
close(1)
end subroutine escribe
```

Control de Fortran, CLI3, curso 2022-2023

Crea co Kate o seguinte arquivo de texto `datos3.txt`, no cal a liña i-ésima ten i números enteiros:

```
1
2 3
4 5 6
7 8 9 8
```

Escribe un programa en Fortran que lea este arquivo a un vector dinámico enteiro **x**. Para isto, conta primeiro o número n de liñas e reserva memoria para **x** con $m = n(n+1)/2$ elementos. Logo, le tódolos números aos elementos de **x** (no arquivo `datos3.txt` hai m valores) e móstrao por pantalla. Chama ao subprograma `matriz(...)`, do tipo e cos argumentos axeitados, que calcule os elementos dunha matriz enteira **a** cadrada de orde m. Para calcular o elemento a_{ij} desta matriz, debes sumar os elementos de **x** ata que esta suma supere o valor ij^2 ou chegues ao final do vector **x**. O valor a_{ij} será o número de elementos sumados. O subprograma debe mostrar por pantalla a matriz **a**, cada fila nunha liña.

```fortran
program cli3
integer,allocatable :: x(:),a(:,:)
open(1,file='datos3.txt',status='old',err=1)
n=0
do
    read (1,*,end=2);n=n+1
end do
2 m=n*(n+1)/2
allocate(x(m),a(m,m))
rewind(1)
l=0
do i=1,n
    read (1,*) (x(l+j),j=1,i)
    l=l+i
end do
print *,'x=',x
close(1)
call matriz(x,m,a)
deallocate(x,a)
stop
1 stop 'datos3.txt non existe'
end program cli3
!-----------------------------------------------------
subroutine matriz(x,m,a)
integer,intent(in) :: x(m),m
integer,intent(out) :: a(m,m)
print *,'a='
do i=1,m
    do j=1,m
        s=0;u=i*j**2;k=0
```

```
        do
            k=k+1;s=s+x(k)
            if(s>u.or.k>m) exit
        end do
        a(i,j)=k
    end do
    print *,(a(i,j),j=1,m)
end do
end subroutine matriz
```

Control de Fortran, CLI4, curso 2022-2023

Escribe un programa en Fortran que lea por teclado un número enteiro n impar entre 5 e 15 (ambos incluidos), volvendo a ler n se non se cumpren estas condicións (usa n=7). Define $m = \lceil n/2 \rceil$ e reserva memoria para unha matriz **a** cadrada real de orde n e un vector **x** real de lonxitude m. Dalle aos elementos a_{ij} valores enteiros aleatorios entre 1 e 10, e mostra por pantalla **a**, cada fila nunha liña da terminal. Logo, chama ao subprograma `vector(...)`, do tipo e cos argumentos axeitados, que calcule o vector **x** de modo que:

$$x_k = \sum_{i=l}^{p} \sum_{j=l}^{p} a_{ij}, \quad k = 1, \ldots, m \tag{1}$$

onde $l = \text{máx}(1, m-k)$ e $p = \text{mín}(m+k, n)$, pero debes rematar de sumar cando o número de elementos a_{ij} sumados supere $2k$. Finalmente, crea un arquivo `datos4.txt` e almacena na súa liña i-ésima, con $i = 1, \ldots, n$, os valores a_{ij} con $j = 1, \ldots, i$. Mostra tamén **x** por pantalla nunha soa liña.

```
program cli4
real,allocatable :: a(:,:),x(:)
do
    print '("5<=n<=15,impar? ",$)'
    read *,n
    if(mod(n,2)==1.and.n>=5.and.n<=15) exit
end do
m=ceiling(n/2.)
allocate(a(n,n),x(m))
call random_number(a)
a=int(9*a+1)
call vector(a,n,x,m)
open(1,file='datos4.txt')
do i=1,n
    write (1,*) (a(i,j),j=1,i)
end do
close(1)
print *,'x=',x
deallocate(a,x)
end program cli4
!-----------------------------------------------------------
subroutine vector(a,n,x,m)
real,intent(in) :: a(n,n)
integer,intent(in) :: m
real,intent(out) :: x(m)
integer :: p
do k=1,m
    s=0;u=2*k;q=0;l=max(1,m-k);p=min(m+k,n)
    bucle: do i=l,p
        do j=l,p
            s=s+a(i,j);q=q+1
            if(q>u) exit bucle
```

```
        end do
    end do bucle
    x(k)=s
end do
end subroutine vector
```

Control de Fortran, CLI5, curso 2022-2023

Escribe un programa en Fortran que lea por teclado un número enteiro n comprobando que sexa maior que 3 (usa n=10). Define unha constante enteira m=4 e dous vectores estáticos enteiros **y** e **z** con m elementos cada un, sendo **z**=(2,3,5,7). Reserva memoria para un vector enteiro **x** con n elementos $x_i = i^2 + i - 1$, con $i = 1, \ldots, n$. Calcula o elemento y_i, con $i = 1, \ldots, m$, chamando repetidamente ao subprograma `calcula_y(...)`, do tipo e cos argumentos axeitados, que calcule y_i definido como:

$$y_i = \sum_{j=1}^{n} j\delta_{ij} z_i, \quad i = 1, \ldots, m \tag{1}$$

onde $\delta_{ij} = 1$ se x_j é múltiplo de z_i e $\delta_{ij} = 0$ en caso contrario. De volta ao programa principal, crea o arquivo `datos5.txt` con m liñas. Na liña i-ésima, con $i = 1, \ldots, m$, almacena s_i, definido como:

$$s_i = \sum_{j=k_i}^{l_i} y_j \tag{2}$$

onde $k_i = \text{máx}(1, i-1)$ e $l_i = \text{mín}(i+1, m)$.

```fortran
program cli5
integer,parameter :: m=4
integer,allocatable :: x(:)
integer :: z(m)=(/2,3,5,7/),y(m),calcula_y
do
    print '("n>3? ",$)'
    read *,n
    if(n>3) exit
end do
allocate(x(n))
forall(i=1:n) x(i)=i**2+i+1
do i=1,m
    y(i)=calcula_y(x,n,z(i))
end do
open(1,file='datos5.txt')
do i=1,m
    k=max(1,i-1);l=min(i+1,m)
    write (1,*) sum(y(k:l))
end do
close(1)
deallocate(x)
end program cli5
!-----------------------------------------------------------
integer function calcula_y(x,n,z) result(y)
integer,intent(in) :: x(n),n,z
y=0
do j=1,n
    if(mod(x(j),z)==0) y=y+j
end do
```

```
y=y*z
end function calcula_y
```

Control de Fortran, CLI6, curso 2022-2023

Escribe un programa en Fortran que declare unha constante enteira n=3 e defina unha matriz estática **a** cadrada de orde n co valores [1 2 3; 4 5 6; 7 8 9]. Mostra por pantalla **a** con formato enteiro de ancho 3, cada fila nunha liña da terminal. Chama ao subprograma `suma_impar(...)`, do tipo e cos argumentos axeitados, que calcule s, definido como:

$$s = \sum_{i,j\in\mathcal{A}} (i+j)a_{ij}, \quad \mathcal{A} = \{(i,j) : a_{ij}\,\%2 = 1, \forall i,j = 1,\ldots,n\} \tag{1}$$

é decir, $\mathcal{A}$ é o conxunto dos índices dos elementos impares de **a**. De volta no programa principal, mostra por pantalla s. Logo, para $i = 1,\ldots,n$, calcula u_i=`sum(a)`$/i$ e s_i, dado por:

$$s_i = \sum_{j=1}^{i} a_{ij} + \sum_{j=i+1}^{n} a_{ji} \tag{2}$$

A suma do lado dereito da ecuación anterior debe deterse cando supere u_i. Crea o arquivo `datos6.txt` e almacena na súa liña i-ésima os valores i, s_i e u_i con $i = 1,\ldots,n$.

```fortran
program cli6
integer,parameter :: n=3
integer :: suma_impar,a(n,n)=reshape((/1,4,7,2,5,8,3,6,9/),shape(a))
print *,'a='
do i=1,n
    do j=1,n
        print '(i3," ",$)',a(i,j)
    end do
    print *,''
end do
s=suma_impar(a,n)
print *,'s=',s
open(1,file='datos6.txt')
sa=sum(a)
do i=1,n
    s=0;u=sa/i
    do j=1,i
        s=s+a(i,j)
        if(s>u) exit
    end do
    if(s<=u) then
        do j=i+1,n
            s=s+a(j,i)
            if(s>u) exit
        end do
    end if
    write (1,*) i,s,u
end do
close(1)
```

```fortran
end program cli6
!-----------------------------------------------------------
integer function suma_impar(a,n) result(s)
integer,intent(in) :: a(n,n),n
s=0
do i=1,n
    do j=1,n
        k=a(i,j)
        if(mod(k,2)==1) s=s+(i+j)*k
    end do
end do
end function suma_impar
```

Control de Fortran, CLI1, curso 2023-2024

Escribe un programa en Fortran chamado `exame.f90` que defina unha constante enteira n=3. Define unha matriz cadrada **a** de orde n, e le por teclado os seus valores. Usa a matriz [1 15 7; 2 11 8; 3 13 9]. Imprime **a** na terminal (unha fila en cada liña). Chama ao subprograma `subprog(...)`, co tipo e argumentos axeitados, que calcule o número m de elementos de **a** que son impares ou maiores que 5. O programa principal debe almacenar no arquivo `cli1.txt` os elementos de **a** ata que a súa suma supere $2m$ sen saírse da matriz.

```fortran
program exame
integer,parameter :: n=3
integer :: subprog,a(n,n)
print *,'a?'
do i=1,n ! 1 15 7; 2 11 8; 3 13 9
    read *,a(i,:)
end do
print *,'a:'
do i=1,n
    print *,a(i,:)
end do
m=subprog(a,n)
!m=sum(pack(a,mod(a,2)==1.or.a>5))  ! vectorizado
print *,'m=',m
open(1,file='cli1.txt')
s=0;m2=2*m
filas: do i=1,n
    do j=1,n
        k=a(i,j)
        write (1,*) k
        s=s+k
        if(s>m2) exit filas
    end do
end do filas
close(1)
end program exame
!---------------------------------------
integer function subprog(a,n) result(m)
integer,intent(in) :: a(n,n),n
m=0
print *,'elementos de a impares ou >5:'
do i=1,n
    do j=1,n
        k=a(i,j)
        if(mod(k,2)==1.or.k>5) m=m+1
    end do
end do
end function subprog
```

Control de Fortran, CLI2, curso 2023-2024

Escribe co editor o seguinte arquivo `cli2.txt`:

```
5
1 2 3 4 5
6 7 8 9 8
7 6 5 4 3
2 1 2 3 4
5 6 7 8 9
```

Escribe un programa en Fortran chamado `exame.f90` que lea o número da primeira liña á variable n, reserve memoria para unha matriz real cadrada de orde n e lea os restantes números á matriz **a**, pechando o arquivo. Chama ao subprograma `subprog(...)`, do tipo e cos argumentos axeitados, que defina m=0 e percorra os elementos a_{ij} de **a**. Se $j < i - 1$, o subprograma debe mostrar 6 espazos en branco. Cando $i - 1 \leq j \leq i + 1$, debe mostrar a_{ij} con ancho 6 e 1 decimal, e incrementar m en 1. Nos dous casos, sen pasar á liña seguinte. So debe pasar á liña seguinte ao final da fila. O programa principal debe mostrar m por pantalla con formato enteiro de anchura mínima.

```fortran
program exame
real,allocatable :: a(:,:)
integer :: subprog
open(1,file='cli2.txt',status='old',err=1)
read (1,*) n
allocate(a(n,n))
do i=1,n
    read (1,*) a(i,:)
end do
close(1)
m=subprog(a,n)
print '("m=",i0)',m
deallocate(a)
stop
1 stop 'erro lendo cli2.txt'
end program exame
!----------------------------
integer function subprog(a,n) result(m)
real,intent(in) :: a(n,n)
integer,intent(in) :: n
m=0
do i=1,n
    k=i-1;l=i+1
    do j=1,n
        if(j<k) then
            print '(a6,$)',''
        else if(j<=l) then
            print '(f6.1," ",$)', a(i,j)
            m=m+1
        end if
    end do
    print *,''
end do
end function subprog
```

Control de Fortran, CLI3, curso 2023-2024

Escribe un programa en Fortran chamado `exame.f90` que lea por teclado un número enteiro n e un vector **x** enteiro de lonxitude n. Usa n=7 e **x**=[8 15 -3 -6 1 9 0]. O programa debe chamar ao subprograma `subprog(...)`, do tipo e cos argumentos axeitados, que debe percorrer **x** sustituindo cada elemento $x_i > n$ por $x_i - n$ e cada elemento $x_i < 1$ por $x_i + n$. O subprograma debe calcular m, o número de valores do primeiro caso ($x_i > n$) e o número k de elementos de **x** que hai que sumar, comezando en x_1 ata superar $10m$ (se chegas ao final de **x**, debes comezar de novo polo principio). O programa principal debe mostrar m e k por pantalla, e almacenar no arquivo `cli3.txt`, en cada liña, i e $mx_i + k$ para $i = 1, \ldots, n$, sendo m o número de valores do primeiro caso ($x_i > n$).

```fortran
program exame
integer,allocatable :: x(:)
print '("n? ",$)'
read *,n
allocate(x(n))
print '("x? ",$)'
read *,x
call subprog(x,n,m,k)
print '("m=",i0," k=",i0)',m,k
open(1,file='cli3.txt')
do i=1,n
    write (1,*) i,m*x(i)+1
end do
close(1)
deallocate(x)
end program exame
!-------------------------------
subroutine subprog(x,n,m,k)
integer,intent(inout) :: x(n)
integer,intent(in) :: n
integer,intent(out) :: m,k
m=0
do i=1,n
    j=x(i)
    if(y>n) then
        x(i)=j-n;m=m+1
    else if(j<1) then
        x(i)=j+n
    end if
end do
i=1;k=0;j=0;l=10*m
do while(j<l)
    j=j+x(i);k=k+1;i=i+1
    if(i>n) i=1
end do
end subroutine subprog
```

Control de Fortran, CLI4, curso 2023-2024

Crea co editor o arquivo `cli4.txt` co seguinte contido:

```
10
3 7 10 1 6 9 8 5 4 2
```

Escribe un programa en Fortran chamado `exame.f90` que lea o número da primeira liña e o almacene en n. Reservan dous vectores enteiros **x** e **y**, ambos de lonxitude n. Le os valores da segunda liña do arquivo ao vector **x**. Chama ao subprograma `subprog(...)`, do tipo e cos argumentos axeitados, que calcule y_i, con $i = 1, \ldots, n$. Para calcular y_i, suma os valores x_j, comezando en j=1, ata que a suma supere $i + 10$. Cando $j > n$, debes continuar en j=1. O elemento y_i será o índice j do último x_j sumado. O programa principal debe imprimir o vector **y** na terminal.

```fortran
program exame
integer,allocatable :: x(:),y(:)
open(1,file='cli4.txt',status='old',err=1)
read (1,*) n
allocate(x(n),y(n))
read (1,*) x
close(1)
call subprog(x,n,y)
print *,'y=',y
deallocate(x,y)
stop
1 stop 'erro lendo cli4.txt'
end program exame
!-----------------------------------
subroutine subprog(x,n,y)
integer,intent(in) :: x(n),n
integer,intent(out) :: y(n)
do i=1,n
    s=0;j=1;k=i+10
    do while(s<k)
        s=s+x(j);j=j+1
        if(j>n) j=1
    end do
    y(i)=j
end do
end subroutine subprog
```

Control de Fortran, CLI5, curso 2023-2024

Escribe un programa en Fortran chamado `exame.f90` que defina dous vectores **x** e **y**, ambos de lonxitude 5, con valores [5 12 9 8 17] e [11 9 7 10 5], respectivamente. Chama ao subprograma `subprog(...)`, que calcule o número m de valores comúns aos vectores **x** e **y** que hai que sumar para que a suma supere 20 (cada valor común súmase so unha vez). O programa principal debe almacenar no arquivo `cli5.txt` tódolos elementos de **x** e **y**, cada par de elementos x_i, y_i nunha liña i distinta con $i = 1, \ldots, n$, e máis o número m de elementos de **x** sumados con formato enteiro de ancho mínimo.

```fortran
program exame
integer,parameter :: n=5
integer :: subprog,x(n)=[5,12,9,8,17],y(n)=[11,9,7,10,5]
m=subprog(x,y,n)
open(1,file='cli5.txt')
do i=1,n
    write (1,*) x(i),y(i)
end do
write (1,'("m=",i0)') m
close(1)
stop
1 stop 'erro open cli5.txt'
end program exame
!-----------------------------------
integer function subprog(x,y,n) result(m)
integer,intent(in) :: x(n),y(n),n
m=0;j=0
do i=1,n
    k=x(i)
    if(any(k==y)) then
        m=m+1;j=j+k
        if(j>20) exit
    end if
end do
end function subprog
```

Control de Fortran, CLI6, curso 2023-2024

Escribe un programa en Fortran chamado `exame.f90` que defina unha constante enteira n=3 e unha matriz cadrada enteira **a** de orde n. O programa debe chamar ao subprograma `subprog(...)`, do tipo e cos argumentos axeitados, que calcule os elementos a_{ij} da matriz **a**, con $i, j = 1, \ldots, n$. O valor a_{ij} debe ser o número de números impares entre i e $(i+j)^2$ que hai que sumar para superar $2i + 3j$. O programa principal debe almacenar no arquivo `cli6.txt` a matriz **a**, unha fila en cada liña do arquivo.

```
program exame
integer,parameter :: n=3
integer :: a(n,n)
call subprog(a,n)
open(1,file='cli6.txt')
do i=1,n
    write (1,*) a(i,:)
end do
close(1)
end program exame
!-------------------------------------
subroutine subprog(a,n)
integer,intent(out) :: a(n,n)
integer,intent(in) :: n
integer :: x,p
do i=1,n
    do j=1,n
        k=0;l=2*i+3*j;x=(i+j)**2
        do m=i,x
            if(k>l) exit
            if(mod(m,2)==1) then
                k=k+m;p=p+1
            end if
        end do
        a(i,j)=p
    end do
end do
end subroutine subprog
```